

MATCHIQ

AI-Driven Talent Intelligence & Role-Fit Automation

Fresh Data-Heavy Project Case Study — v4.0

Decision architecture • process engineering • deterministic intelligence • selective AI • low-code orchestration • governance • change management

Document attribute	Current record
Project	MatchIQ
Document version	3.3 — data-heavy working master, updated 17 September 2026
Date	17 September 2026
Primary audience	Hiring managers and interview panels for AI Governance, Digital Transformation, AI Product, AI Automation and AI-enabled Process Transformation
Business utility	Automated candidate-to-job intelligence: discover, screen, prioritise, analyse, assess and persist role-fit evidence
Design thesis	The product is the decision architecture; Python, n8n, Groq, documents and spreadsheets are implementation evidence
Current active semantic components	Agent 2 — CV Intelligence; Agent 3 — Role Fit
Current deterministic components	Job discovery/scraping, scoring, filtering, JD Intelligence, candidate evidence selection, identity, persistence and state control
Current orchestration	n8n 2.38.7 in the exported workflow
Current model recorded	openai/gpt-oss-120b via Groq
Current controlled AI budget	20 jobs after deterministic ranking/Limit
Important evidence boundary	The complete current jd_intelligence.py and complete final Agent 2 / Agent 3 Groq prompt archives are not present in the exported workflow alone; where necessary this document labels them as development-record evidence rather than pretending they are source-controlled artefacts

Core proposition: reduce a noisy job market to a controlled decision set before spending expensive reasoning capacity.

1. Executive Summary

MatchIQ was developed as an automation and decision-support system for a practical problem: a candidate can encounter hundreds or thousands of apparently relevant vacancies, but only a much smaller number deserve detailed human attention. The system therefore does not attempt to make every job 'AI analysed'. It first makes deterministic decisions about relevance, eligibility, duplication, ranking and resource allocation. Only then does it use AI for the parts that benefit from semantic judgement.

The result is a controlled pipeline that can be explained to a non-technical stakeholder in one sentence: MatchIQ takes a live candidate profile and a changing job market, filters and prioritises the opportunities using explicit rules, uses AI selectively to understand the role and candidate evidence, compares the two, and stores an auditable result for human review.

The project became materially stronger through failure. A first implementation of the 8–10-year criterion was too literal and reduced 1,313 discovered jobs to five selected jobs. A separate run generated 1,470 requests and repeated HTTP 429 responses, exposing the cost of enriching every discovered job. The project then separated job eligibility from candidate-role fit, retained the raw experience wording, reduced unnecessary model-facing work, introduced serial execution and pacing, cached candidate evidence, and moved JD intelligence towards deterministic Python.

The architecture also evolved from three conceptual AI agents to two active LLM components. The JD interpretation function was retained as a logical responsibility but moved out of the LLM layer because the work was structured and repeatable. This is a key differentiator: the system became more sophisticated by becoming less AI-heavy.

For a hiring manager, the project demonstrates analytical thinking, process redesign, product thinking, low-code automation, Python development, AI integration, failure analysis, data lineage, resource governance and change management in one continuous artefact.

1.1 The business story in the first two minutes

The problem is not 'how to use an LLM to read a CV'. The problem is how to turn a changing, noisy labour market into a repeatable decision process. A human can read a job description, read a CV and make a judgement. At scale, the same person cannot do that consistently across hundreds of opportunities without prioritisation, repeated work and loss of traceability.

MatchIQ addresses that operating problem in layers. The candidate is converted into a reusable evidence profile. Jobs are collected and represented as structured records. Deterministic logic handles hard constraints and broad relevance. A bounded AI budget is then allocated to the strongest opportunities. AI outputs are validated and persisted with explicit processing states. A human remains the final decision-maker.

The sophistication is therefore in the allocation of responsibility: deterministic code handles repeatable facts; AI handles semantic interpretation; n8n handles sequencing and integration; persistent data structures retain the evidence needed to understand what happened.

2. Starting Point: Candidate DNA

The project began with the idea that a CV should not be represented simply as a job title plus keywords. A candidate's value is distributed across functions, capabilities, domains, governance experience, leadership, delivery context, tools and outcomes. This became the Professional DNA concept.

Professional DNA dimension	Baseline weighting	Decision meaning
AI Governance	18%	Governance, responsible AI, policy and control relevance
AI / GenAI / Automation	12%	AI-enabled transformation and automation relevance
Regulatory / Compliance	15%	Regulated-environment and compliance relevance
Risk / Controls / Assurance	15%	Risk management, control and assurance relevance
Transformation / Operating Model	15%	Change, operating-model and transformation relevance
Programme / Product / Delivery	10%	Delivery and product/programme execution relevance

Professional DNA dimension	Baseline weighting	Decision meaning
Financial Services / FinCrime	10%	Financial-services and financial-crime domain relevance
Stakeholder / Cross-Border	5%	Senior stakeholder, international and cross-border context

The model was deliberately multi-dimensional because hybrid roles can be poorly represented by a single title. A role may be described as AI Governance, but the actual work may include operating-model transformation, risk controls, product delivery, regulatory engagement or programme leadership. Conversely, a role with an attractive AI title may contain little of the work the candidate actually wants to perform.

The Professional DNA model therefore became the reference vocabulary for deterministic job scoring and later role-specific assessment. The project framework records the separation between 'opportunity relevance' and 'candidate-role fit' as a foundational product decision. (Source: MatchIQ project framework.)

2.1 Hybrid-role and functional-keyword mapping

A key product requirement was to avoid title-only matching. The deterministic layer uses functional and thematic signals such as AI Governance, AI/GenAI/Automation, Regulatory/Compliance, Risk/Controls/Assurance, Transformation/Operating Model, Programme/Product/Delivery, Financial Services/FinCrime and Stakeholder/Cross-Border. These are not treated as interchangeable synonyms; they are separate evidence dimensions.

Signal family	Examples recorded in the project	Why it matters for hybrid roles
Governance	AI Governance, governance actions, controls, assurance	Captures roles where governance is a function rather than the title
AI / automation	AI, GenAI, automation	Captures digital-transformation roles with AI embedded in delivery
Regulation	Regulatory, compliance, regulatory frameworks	Separates regulated accountability from generic process work
Risk	Risk, controls, assurance	Captures second-line / control-oriented responsibilities
Transformation	Transformation, operating model	Captures change roles whose titles may not contain 'AI'
Delivery	Programme, product, delivery	Captures execution ownership in hybrid roles
Domain	Financial services, FinCrime	Adds domain context to otherwise generic governance roles
Stakeholders	Stakeholder, cross-border	Captures roles where influence and international delivery are central

3. Product Logic: Two Decisions, Not One Score

The product deliberately separates two questions. The first is whether a vacancy deserves deeper attention. The second is whether the candidate fits that specific vacancy. Combining them into one score creates an opaque system in which a job can appear highly relevant while still being a poor candidate fit, or vice versa.

DECISION 1 – OPPORTUNITY RELEVANCE

Is this job sufficiently relevant/actionable to justify deeper analysis?

DECISION 2 – CANDIDATE-ROLE FIT

Given this exact role, how strongly does the candidate evidence support the requirements?

Final decision support:

Proceed / Prioritise / Monitor / Reject

↓

Human judgement remains authoritative

Decision layer	Evidence used	Mechanism	Output
Discovery	Search clusters, locations, job-market sources	Python	Raw opportunity population

Decision layer	Evidence used	Mechanism	Output
Eligibility	Role family, exclusions, seniority, location, domain, recency and configured constraints	Python rules	Eligible / excluded
Relevance	Professional DNA signals, semantic similarity, title/bonus/penalty signals, market signals	Python + deterministic scoring	Match Score
Resource allocation	Opportunity Gate / ranking / AI budget	Python + n8n	AI-eligible population
Candidate evidence	CV text, version, explicit experience	Agent 2 + cache	Candidate Evidence Profile
Role intelligence	JD content, role purpose, responsibilities, requirements	Deterministic JD Intelligence; earlier Agent 1 concept	Structured role object
Role fit	Role requirements + selected candidate evidence	Agent 3	Validated Role Fit
Decision support	Role Fit + evidence/gaps + operational status	Human review	Action

4. Architecture Evolution: From Concept to Controlled Workflow

Stage	What existed	Problem exposed	Design response	Why the change matters
Concept	Candidate DNA + job matching	One broad match concept could hide different decisions	Separate opportunity relevance from candidate-role fit	Clear product semantics
Three-agent prototype	Agent 1 JD + Agent 2 CV + Agent 3 fit	Three LLM calls created duplicated context and cost	Retain logical responsibilities but remove unnecessary LLM work	Lower model dependency
Provider transition	OpenAI-compatible provider approach; later Groq	Provider limits constrained repeated calls	Move working inference to Groq / openai/gpt-oss-120b	Provider substitution without redesigning business logic
Token-pressure stage	Large job population entered agentic work	TPM/token/day constraints became operational bottlenecks	Deterministic filtering, AI budget, serialisation, waits, context discipline	Treat inference as a governed resource
Experience-rule correction	Literal '8–10 years' wording rule	1,313 discovered → 5 selected; market representation became implausibly narrow	Separate job requirement wording from candidate target seniority	Preserve recall and semantic meaning
Opportunity Gate correction	Second n8n threshold ≥40	Only 1/60 passed after upstream screening	Remove redundant rejection layer; retain score as data	Avoid control without decision value
JD agent retirement	LLM Agent 1	Repeatable JD transformation did not justify model consumption	Deterministic Python JD Intelligence + parser	More deterministic and cheaper
CV caching	CV analysed repeatedly	Same candidate context was repeatedly regenerated	Versioned candidate cache + conditional branch	Reuse work until source changes
Agent 3 transport redesign	Native agent / structured-output path	Tool-call/parser failure and missing required field	Direct Groq HTTP + strict JSON + parser in development record	Separate transport from validation
Mutable source	Static PDF orientation	Role-type and CV changes required source-level maintainability	Move towards Word / mutable document source	Change at source, then propagate

The project framework describes this evolution as a controlled agentic workflow: deterministic pre-screening, deduplication, an agentic budget, explicit statuses, stable Job Key, validated Role Fit and persistent master data. (Source: MatchIQ project framework.)

5. Failure-Driven Engineering: The Data That Changed the Design

5.1 The 1,313 → 5 experience-filter failure

Country	Discovered	Selected	Observed implication
Luxembourg	87	0	No selected jobs under the literal rule
Germany	415	0	Large discovered population but zero survivors

Country	Discovered	Selected	Observed implication
Italy	257	0	Large discovered population but zero survivors
Spain	190	1	Small number survived
Switzerland	174	3	Largest selected count despite smaller population
Poland	190	1	Small number survived
TOTAL	1,313	5	99.62% of discovered jobs did not survive the selection logic

The observed selection rate was $5 / 1,313 =$ approximately 0.38%, meaning approximately 99.62% of discovered jobs were removed. The development diagnosis was that the system had effectively converted 'candidate target seniority around 8–10 years' into 'the employer must literally state an equivalent 8–10-year phrase'. That was a logic error, not merely a tuning issue.

INCORRECT SEMANTIC INTERPRETATION

Candidate target: ~8–10 years



Job text must literally contain "8–10 years"



Most legitimate senior roles rejected

CORRECTED PRODUCT DISTINCTION

Job eligibility:

What seniority does the role require?

Candidate-role fit:

How well does the candidate evidence support that requirement?

The development record proposed preserving the raw experience wording and deriving structured fields such as experience_requirement_raw, experience_min_years, experience_max_years, experience_type, experience_match and experience_evidence. This makes the original wording auditable rather than destroying it during filtering. (Source: MatchIQ development record.)

5.2 The 1,470-request / 429 operational failure

1,313 discovered



~1,313 detail-enrichment requests



1,470 requests observed in the run



repeated HTTP 429 responses



only a small subset reached successful downstream evaluation

This run demonstrated that the bottleneck was not merely model output. The system was paying an acquisition and enrichment cost before it could even make the AI-budget decision. That changed the optimisation question from 'How do we make the agents smaller?' to 'How do we stop unnecessary work entering the expensive path in the first place?'

Problem	Root cause	Process response
Too many detail requests	Every discovered opportunity was enriched	Reduce downstream population and use controlled enrichment
429 responses	Request volume/rate exceeded the observed endpoint tolerance	Serialisation, pacing and bounded retries
Low final count	Experience filter was semantically over-restrictive	Separate eligibility from fit
Unrepresentative country distribution	Upstream rules and enrichment failures distorted the sample	Treat run output as a processing result, not a market truth

6. Deterministic Job Intelligence

The deterministic layer is the first major engineering control. It converts a noisy job population into structured, comparable records before an LLM is asked to reason about anything.

Deterministic function	Recorded implementation concept	Output / decision
Job discovery	Search across configured countries and city anchors	Raw job population
Normalisation	Standardise fields and preserve source provenance	Stable job records
Deduplication	LinkedIn URL preferred; company/role/location fallback	Unique opportunities
Professional DNA scoring	Weighted functional and domain signals	Match Score
Market signals	Regulatory markers, governance actions, talent-shortage indicators, international-hiring and scarce-specialist signals	Market/context fields
Hard constraints	Role family, exclusions, location/mobility, domain, recency and configured experience logic	Eligibility
Opportunity ranking	Sort descending by Match Score in n8n	Priority order
AI budget	Limit 20 in the current exported workflow	Bounded agentic population

The current workflow export confirms a Sort node ordered by 'Match Score' descending followed by a Limit node with maxItems = 20. (Source: MatchIQ project framework.)

6.1 Country-level targeting

The project changed final selection from city-level quotas to country-level targeting while retaining city-level search anchors. This was a deliberate volume-control decision. Searching several cities can surface the same opportunity multiple times; country-level final targeting reduces duplicate downstream processing while preserving geographical discovery coverage.

Control	Original tendency	Changed approach	Business reason
Discovery anchor	City-level search	Retain city anchors	Preserve search coverage
Final quota	City-level selection	Country-level final targeting	Reduce duplicate opportunities
Target volume	Higher combined city volume	Approximately 10 final jobs per country in the development design	Bound downstream work

7. Opportunity Gate: Resource Allocation Rather Than Another Match Score

The Opportunity Gate was conceived as the control point between deterministic screening and expensive agentic analysis. Its purpose is not to decide whether a candidate should be hired. Its purpose is to decide whether an opportunity deserves deeper analytical expenditure.

```

DISCOVERY
↓
DETERMINISTIC INTELLIGENCE
  Match + Market + Role dimensions
↓
OPPORTUNITY GATE / RESOURCE ALLOCATION
  "Is this worth deeper investigation?"
↓
AGENTIC INTELLIGENCE
  Candidate evidence + role intelligence + fit
↓
FINAL DECISION SUPPORT
  Apply / Prioritise / Monitor / Reject

```

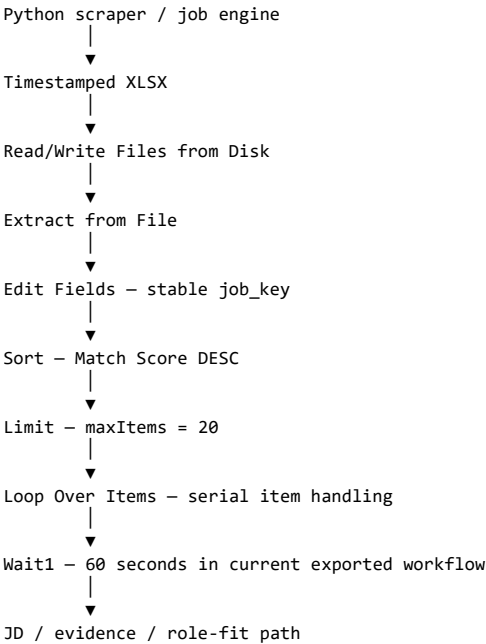
Score / signal	Interpretation	Control status
Match Score	How closely the job resembles the Professional DNA target	Ranking / relevance
Opportunity Gate Score	Whether the opportunity is worth deeper reasoning capacity	Recorded as a data signal; redundant downstream rejection was removed after 1/60 test
Talent Shortage Score	Market signal related to scarcity	Supporting signal
Deterministic Evidence Score	Retrieval/evidence indicator for downstream assessment	Not the final candidate fit score

The development record originally described a threshold of Opportunity Gate Score ≥ 40 . Testing produced 1 pass out of 60. The subsequent decision was to remove the additional n8n rejection because upstream filtering had already constrained the population. The score remained available as information rather than being allowed to create an unnecessary second rejection point. (Source: MatchIQ development record.)

This is a useful process-engineering distinction: a control is only valuable if it improves the decision enough to justify the cost and risk it introduces.

8. How 60 Becomes 20

The current exported n8n workflow contains the following explicit sequence: Python job run → XLSX read → extraction → field normalisation → Sort by Match Score descending → Limit to 20 → Loop Over Items → Wait → downstream assessment.



Stage	Population / state	Why it exists
Selected job workbook	60 in the controlled architecture	Representative deterministic set available to the workflow
Sort	60 ordered by Match Score	Use scarce AI capacity on the strongest deterministic opportunities first
Limit	20	Explicit AI-budget boundary
Loop	1 item at a time	Avoid concurrent bursts
Wait	60 seconds in the current exported workflow	Pace calls and reduce provider pressure

The 60 → 20 reduction is a 66.7% reduction in model-facing population relative to the 60-job set. It is not legitimate to claim that this equals a 66.7% token saving, because token usage depends on prompt size and the actual context sent per job.

The project record also discusses a target 20 → 10 deterministic evidence filter. That remains a target unless execution evidence proves it has been implemented. The correct project-document discipline is to label it as target rather than completed.

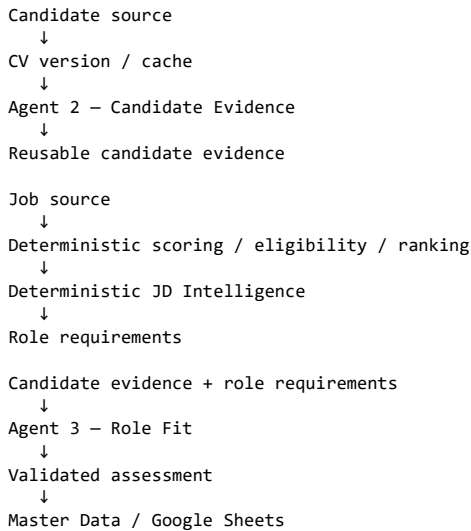
9. Agent Architecture: From Three Logical Agents to Two Active LLM Components

Logical responsibility	Initial implementation	Current treatment	Reason
Job / JD intelligence	Agent 1 — LLM	Deterministic Python + parser	Repeatable extraction did not justify an LLM call

Logical responsibility	Initial implementation	Current treatment	Reason
Candidate intelligence	Agent 2 — LLM	Active + versioned cache	Semantic candidate evidence is reusable
Role Fit	Agent 3 — LLM	Active; Groq HTTP path in development record	Semantic comparison is the principal AI judgement

The reduction from three to two is not a reduction in business logic. It is a reduction in unnecessary generative computation. The logical functions remain visible, but the implementation mechanism is chosen according to the nature of the work.

FINAL LOGICAL ARCHITECTURE



10. Agent 2 — CV Intelligence

Agent 2 converts the CV into a reusable Candidate Evidence Profile. The central control is vacancy independence: the CV is analysed without a job description so that the resulting evidence can be reused across many vacancies without being biased by whichever role happened to be processed first.

The current exported workflow shows a Google Docs source feeding Agent 2, followed by an Upsert into the MatchIQ_Candidate_Profile data table. The recorded Agent 2 prompt explicitly requires evidence-only extraction, no fit assessment, no scoring, no job recommendation, non-duplication and a compact output ideally under 1,000 tokens.

Prompt control	Current instruction / design	Why it matters
Evidence boundary	Use only information explicitly stated in the CV	Prevents inferred capability
Vacancy independence	Do not assess fit for any job	Makes profile reusable
No scoring	Do not score candidate	Keeps candidate evidence separate from role fit
No recommendation	Do not recommend jobs	Avoids premature decision-making
Structured headings	Ten fixed headings	Predictable downstream parsing / retrieval
Non-duplication	Remove repeated evidence	Controls context size
Output budget	Ideally under 1,000 tokens	Controls repeated downstream context

10.1 Current Agent 2 prompt recorded in the workflow

=Analyze the following CV for MatchIQ.

```
CV TEXT:
{{ $('Get a document').item.json.content }}
```

Create a compact reusable Candidate Evidence Profile for later role-fit assessment.

Use ONLY information explicitly stated in the CV. Never infer or invent capabilities, technologies, responsibilities, authority, qualifications, certifications, budget ownership, people management, or technical experience.

Do not assess fit for any job.
Do not score the candidate.
Do not recommend jobs.

Extract only high-value, non-duplicated evidence under these headings:

PROFILE

2-3 concise lines covering career progression, professional focus, and seniority.

CORE EXPERTISE

6-10 strongest explicitly evidenced functional capabilities.

CAREER EVIDENCE

8-12 strongest roles, responsibilities, programmes, and projects. Prioritize ownership, scope, and distinctive experience.

DOMAIN EVIDENCE

5-8 evidenced industries, business domains, regulatory environments, or organisational contexts.

LEADERSHIP & STAKEHOLDERS

5-8 evidenced examples of leadership, programme/product ownership, cross-functional delivery, senior stakeholder engagement, international delivery, or change leadership. Distinguish leading, owning, coordinating, contributing, and participating.

TECHNICAL & DIGITAL

8-12 explicitly evidenced technologies, platforms, AI/data capabilities, automation tools, and methodologies.

GOVERNANCE & REGULATORY

5-8 explicit examples of AI governance, risk, compliance, controls, assurance, regulatory frameworks, or operating models.

QUANTIFIED ACHIEVEMENTS

5-8 strongest measurable outcomes. Preserve the original figures.

SENIORITY

2-3 concise lines describing the level of responsibility explicitly demonstrated.

EVIDENCE GAPS

Up to 5 important areas where the CV provides limited or no explicit evidence. Do not treat absence of evidence as proof that the candidate lacks the capability.

Rules:

- Prefer evidence over narrative.
- Preserve exact numbers, role titles, industries, technologies, frameworks, and certifications.
- Remove duplicates.
- Do not convert participation into ownership.
- Do not convert a team size into people management.
- Do not infer cloud, cybersecurity, security architecture, MLOps, hands-on engineering, or budget ownership unless explicitly stated.
- If evidence is ambiguous, omit it.
- Return plain text.
- Use exactly the ten headings above.
- Keep the complete profile concise, ideally under 1,000 tokens.

10.2 Agent 2 output structure

PROFILE

CORE EXPERTISE

CAREER EVIDENCE

DOMAIN EVIDENCE

LEADERSHIP & STAKEHOLDERS

TECHNICAL & DIGITAL

GOVERNANCE & REGULATORY

QUANTIFIED ACHIEVEMENTS

SENIORITY

EVIDENCE GAPS

10.3 Candidate cache and invalidation

Google Docs CV



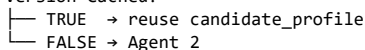
CV Version



Get Candidate Profile Cache



CV Version Cached?



Candidate Profile Upsert

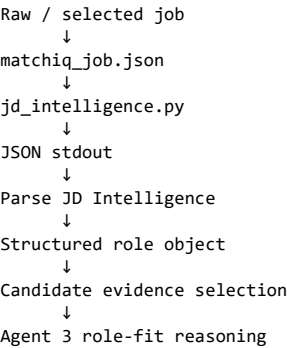


reuse downstream

The current workflow contains a Candidate Profile data table keyed by candidate_id and stores candidate_profile plus updated_at. The workflow record also contains version-based cache logic in the broader project design. The governing principle is simple: unchanged source should not trigger repeated semantic analysis; changed source should invalidate the cache.

11. Deterministic JD Intelligence and Parser Separation

One of the most important architectural decisions was to remove the JD interpretation function from the LLM layer. The JD still needs to be understood, but much of the required transformation is deterministic: cleaning, normalising, extracting structured fields, scoring and passing a stable object downstream.



The current exported workflow executes `jd_intelligence.py` using the command shown below and then parses stdout as JSON. This is a strong interface boundary: the Python script can change internally provided its output contract remains stable.

```
python "C:\Users\Susmita\OneDrive\Desktop\Match IQ\jd_intelligence.py" < "C:\Users\Susmita\OneDrive\Desktop\Match IQ\matchiq_job.json"

const raw = $json.stdout;

if (!raw) {
  throw new Error('Deterministic JD Intelligence returned no stdout.');
```

```
}

let parsed;

try {
  parsed = JSON.parse(raw);
} catch (error) {
  throw new Error(
    'Could not parse deterministic JD Intelligence output as JSON: ' +
    error.message
  );
}

if (parsed.error) {
  throw new Error(
    parsed.error_message || 'Deterministic JD Intelligence failed.'
  );
}

return {
  json: parsed
};
```

The parser is intentionally a separate node rather than being buried inside the JD script. This gives the workflow an explicit contract-validation boundary. If the script emits invalid JSON, the workflow fails at a named stage rather than silently passing malformed data downstream.

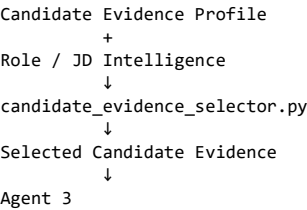
11.1 What the JD intelligence function is responsible for

Responsibility	Purpose	Why deterministic
Clean / normalise JD	Create a stable representation	Repeatable transformation
Extract role fields	Expose role/company/location and other structured information	Stable data mapping
Identify role requirements	Produce role-specific structured fields for later assessment	Avoid unnecessary generation
Retain source evidence	Keep the original role context available	Traceability
Expose machine-readable JSON	Create an interface to downstream n8n nodes	Schema stability

The complete current `jd_intelligence.py` source is not present in the supplied workflow export. This document therefore does not fabricate or reproduce a source file that was not supplied.

12. Candidate Evidence Selection: Context Reduction Before AI

The Candidate Evidence Selector is another important control. Even a good Candidate Evidence Profile can be too large to send in full to every role assessment. The selector is therefore positioned after deterministic JD Intelligence and before Agent 3.



The current exported workflow executes `candidate_evidence_selector.py` from `n8n`, then parses its `stdout` as `JSON`. This means the system can reduce the context before the LLM sees it. That is a more disciplined token optimisation than simply asking the model to ignore irrelevant information.

```
python "C:\Users\Susmita\OneDrive\Desktop\Match IQ\candidate_evidence_selector.py" < "C:\Users\Susmita\OneDrive\Desktop\Match IQ\matchiq_candidate_evidence.json"

const raw = $json.stdout;

if (!raw) {
  throw new Error(
    'Candidate Evidence Selector returned no stdout.'
  );
}

let parsed;

try {
  parsed = JSON.parse(raw);
} catch (error) {
  throw new Error(
    'Could not parse Candidate Evidence Selector output as JSON: ' +
    error.message
  );
}

if (parsed.error) {
  throw new Error(
    parsed.error_message ||
    'Candidate Evidence Selector failed.'
  );
}

return {
  json: parsed
};
```

13. Agent 3 — Role Fit: The Semantic Decision Layer

Agent 3 is the component that genuinely requires contextual reasoning. It receives role-specific requirements and selected candidate evidence and must determine whether the evidence supports the actual work the employer needs done.

The exported workflow contains a native Agent 3 configuration with a structured output parser. The subsequent development record documents a later direct Groq HTTP implementation with a strict JSON schema and an explicit requirement for `executive_summary`. The two should not be conflated: the current exported JSON is the evidence of the exported workflow, while the direct HTTP configuration is evidenced by the dated development record.

Agent 3 assessment dimension	Purpose
Overall fit score	0–100 summary of evidence against this specific role
Fit level	Human-readable interpretation of the score
Recommendation	Action-oriented next step
Overall hiring assessment	Narrative synthesis
Skills / tools mapping	Requirement-by-requirement comparison
Language mapping	Explicit language requirements vs evidence
Domain mapping	Direct and transferable domain evidence
Functional responsibility mapping	Actual work/responsibility comparison

Agent 3 assessment dimension	Purpose
Transferable experience	Credible adjacent evidence
Gaps / risks / evidence gaps	Separate genuine gaps from missing evidence
Hiring manager view	Shortlist logic and interview validation
Executive summary	Concise decision-facing summary

13.1 Current native Agent 3 prompt in the exported workflow

=Assess the candidate's fit for this specific role using ONLY the Job Intelligence and Selected Candidate Evidence below.

JOB INTELLIGENCE

Role:

{{ \$json.role }}

Company:

{{ \$json.company }}

Location:

{{ \$json.location }}

Role Purpose:

{{ \$json.jd_role_purpose }}

Top Responsibilities:

{{ \$json.jd_top_responsibilities.join('\n- ') }}

Required Capabilities:

{{ \$json.jd_required_capabilities.join('\n- ') }}

Seniority and Leadership:

{{ \$json.jd_seniority_leadership }}

Assessment Criteria:

{{ \$json.jd_assessment_criteria.join('\n- ') }}

SELECTED CANDIDATE EVIDENCE

{{ \$json.selected_candidate_evidence }}

DETERMINISTIC EVIDENCE SIGNALS

Matched Job Signals:

{{ \$json.matched_job_signals.join(', ') }}

Deterministic Evidence Score:

{{ \$json.deterministic_evidence_score }}

Deterministic Signal Coverage:

{{ \$json.deterministic_signal_coverage }}%

ASSESSMENT RULES

Assess the candidate against the actual requirements of this specific role.

Use only evidence explicitly supported by the Selected Candidate Evidence.

Distinguish clearly between:

- demonstrated evidence
- transferable experience
- evidence gaps
- genuine risks

Do not treat an absence of CV evidence as proof that the candidate lacks a capability.

Do not invent experience, technologies, certifications, responsibilities, authority, or qualifications.

Do not infer cloud, cybersecurity, security architecture, MLOps, threat modelling, incident response, or hands-on engineering experience unless explicitly evidenced.

Evaluate both direct fit and transferable fit.

Be balanced. Do not over-penalize transferable experience, but do not treat transferable experience as equivalent to direct hands-on experience.

For certifications, distinguish between security certifications required by the role and other certifications actually present in the candidate evidence.

The deterministic evidence score and signal coverage are retrieval indicators only. Do not treat them as the candidate's final fit score.

OUTPUT RULES

Return only the structured output required by the Output Parser.

Do not use Markdown.

Do not use **, *, #, headings with Markdown formatting, or Markdown code fences.

Keep every text value concise and professional.

Do not repeat the same evidence across multiple fields.

SCORING

overall_fit_score:
0-39 = Poor fit
40-59 = Limited fit
60-74 = Partial fit
75-89 = Good fit
90-100 = Excellent fit

The score must reflect the actual evidence and importance of the role requirements.

fit_level must correspond to the score range.

recommendation must be one of:
- Proceed to human review
- Proceed with targeted validation
- Do not prioritize

strongest_fit_evidence:
Provide exactly 3 concise evidence-based points.

key_gaps_and_risks:
Provide exactly 3 important gaps or risks relevant to this specific role.

transferable_experience:
Provide exactly 2 areas where existing experience can reasonably transfer to this role.

evidence_gaps:
Provide up to 3 important areas where the CV provides limited or no explicit evidence.

assessment_summary:
Provide one concise paragraph of approximately 60-100 words explaining the overall assessment, balancing direct evidence, transferable experience, and evidence gaps.

13.2 Development-record Agent 3 Groq HTTP configuration

The dated development record describes a direct HTTP implementation used after the native structured-output path failed. The recorded target configuration was: openai/gpt-oss-120b, temperature 0.2, reasoning effort low, max_completion_tokens 3500, response_format json_schema, strict true. The failure being corrected was a missing executive_summary field in a response subject to a 2,200 completion-token limit. (Source: MatchIQ development record.)

POST
<https://api.groq.com/openai/v1/chat/completions>

model:
openai/gpt-oss-120b

temperature:
0.2

reasoning_effort:
low

max_completion_tokens:
3500

response_format:
json_schema

strict:
true

13.3 Development-record output contract

Required top-level fields:
overall_fit_score
fit_level
recommendation
overall_hiring_assessment
skills_tools_mapping
language_mapping
domain_mapping
functional_responsibility_mapping
transferable_experience
gaps_risks_evidence_gaps
hiring_manager_view
executive_summary

The development record explicitly instructed the model to include executive_summary as the final field, keep it to 80-120 words, and return JSON only. This is an example of prompt-level output control being paired with schema-level validation rather than relying on natural-language instructions alone. (Source: MatchIQ development record.)

14. n8n Node-by-Node Functional and Technical Mapping

The exported workflow contains 32 nodes. The following map explains why each node exists, what it does technically, and what business control it represents. This is intentionally detailed because the workflow itself is evidence of process design.

Node	Type	Technical function	Functional/business reason	If removed
When clicking 'Execute workflow'	manualTrigger	Starts the controlled run.	Allows a deliberate operator-triggered execution.	No controlled run entry point.
Get a document	googleDocs	Retrieves the live CV source document.	Makes the candidate source changeable at source.	No live CV source.
Agent 2 - CV Intelligence	langchain agent	Creates vacancy-independent candidate evidence.	Converts unstructured CV content into reusable evidence.	No reusable semantic candidate layer.
Upsert row(s)	dataTable	Stores candidate profile by candidate_id.	Makes candidate intelligence persistent and reusable.	CV would be reprocessed.
Execute Command	executeCommand	Runs new.py / job engine.	Automates job discovery and deterministic scoring.	No automated market refresh.
Read/Write Files from Disk	readWriteFile	Reads timestamped XLSX output.	Transfers the deterministic job dataset into n8n.	No job dataset handoff.
Extract from File	extractFromFile	Converts XLSX into workflow items.	Turns rows into processable records.	n8n cannot route individual jobs.
Edit Fields	set	Creates stable job_key from LinkedIn URL or fallback.	Establishes record identity and deduplication basis.	Repeat runs risk duplicates.
Sort	sort	Orders Match Score descending.	Allocates limited AI capacity to strongest deterministic opportunities.	AI budget would be arbitrary.
Limit	limit	Sets maxItems = 20.	Explicitly bounds the model-facing population.	Potentially unbounded AI workload.
Loop Over Items	splitInBatches	Processes items serially.	Controls request concurrency.	Burst requests.
Wait1	wait	60-second delay in exported workflow.	Paces the downstream process.	Higher provider-rate exposure.
Edit Fields1	set	Maps workbook fields into stable internal names.	Creates the interface expected by downstream nodes.	Field-name drift / fragile mappings.
Convert to File	convertToFile	Creates JSON handoff file.	Bridges n8n and deterministic JD script.	No script input file.
Read/Write Files from Disk1	readWriteFile	Writes matchiq_job.json.	Provides deterministic script input.	JD script lacks controlled input.
Deterministic JD Intelligence	executeCommand	Runs jd_intelligence.py.	Moves repeatable JD work out of the LLM layer.	Unnecessary AI consumption.
Parse JD Intelligence	code	Parses stdout JSON and throws on errors.	Creates an explicit contract boundary.	Malformed role data could propagate.
Get row(s)	dataTable	Retrieves candidate cache.	Supplies reusable candidate evidence.	Candidate context may be unavailable.
Merge	merge	Combines role and candidate-side paths.	Creates a unified assessment context.	Inputs remain fragmented.
Convert to File1	convertToFile	Serialises candidate evidence.	Prepares selector input.	No controlled file handoff.
Read/Write Files from Disk2	readWriteFile	Writes matchiq_candidate_evidence.json.	Provides selector input.	Selector cannot run against the current object.
Execute Command2	executeCommand	Runs candidate_evidence_selector.py.	Reduces candidate context deterministically.	More candidate tokens reach Agent 3.
Parse Candidate Evidence	code	Parses selector stdout JSON.	Creates validated downstream candidate evidence object.	Invalid selector output could propagate.
Agent 3 - Role Fit	langchain agent	Role-fit reasoning in exported workflow.	Produces semantic assessment.	No candidate-role synthesis.
Groq Chat Model	lmChatGroq	Supplies Groq model to native Agent 3.	Provides model inference configuration.	Agent 3 lacks model.
Structured Output Parser	outputParserStructured	Defines native Agent 3 structured contract.	Attempts deterministic field extraction.	Downstream output is less predictable.
Upsert row(s)1	dataTable	Persists role assessment by job_key.	Creates durable master record.	Results disappear with run.
Append row in sheet	googleSheets	Appends master data to Google Sheet.	Creates an accessible operational review surface.	Human review requires opening internal data store.
Agent 1 – JD Intelligence	langchain agent	Original JD LLM concept; currently disconnected in exported connections.	Historical architecture evidence.	Would reintroduce redundant LLM work if reconnected.
Groq - Agent 1	lmChatGroq	Model attached to retired Agent 1.	Historical configuration evidence.	No role if Agent 1 remains retired.
Groq - Agent 2	lmChatGroq	Model attached to Agent 2.	Current candidate semantic inference path.	No model for Agent 2.

15. Critical Data Mappings

The value of the workflow is not just that nodes exist; each mapping establishes an interface between business meaning and technical representation.

Source field	Internal field	Why mapped	Downstream dependency
LinkedIn URL	job_key	Stable identity preferred over transient row position	Master Data upsert / deduplication
Company	company	Normalise source naming	JD processing / output
Role	role	Stable role label	JD and role-fit prompts
Location	location	Eligibility and human review	Scoring / output
Search Cluster	search_cluster	Preserve discovery rationale	Analytics / provenance
Search Query	search_query	Preserve discovery provenance	Analytics / debugging
Date Posted	date_posted	Recency control	Eligibility / review
Match Score	scraper_match_score	Preserve deterministic ranking signal	Audit / ranking
Opportunity Gate Score	opportunity_gate_score	Preserve gate signal even after redundant rejection removed	Analysis / calibration
Talent Shortage Score	talent_shortage_score	Preserve market signal	Decision context
Full Job Description	full_job_description	Retain source context for JD intelligence	JD script
candidate_id	candidate_id	Stable candidate identity	Candidate cache
cv_version	cv_version	Detect source change	Cache invalidation

15.1 Why stable Job Key matters

Preferred:
job_key = LinkedIn URL

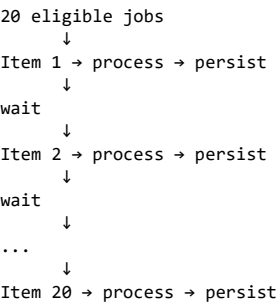
Fallback:
job_key = Company | Role | Location

This is more than a technical convenience. Without a stable identity, the same vacancy can be treated as a new business opportunity on every run, leading to repeated AI analysis, duplicated records and distorted market counts. The Job Key is therefore both a data-governance control and a token-optimisation control.

16. Why We Loop the Data and Why We Upsert Rows

16.1 Looping

Loop Over Items is used because the AI/provider path is a resource-constrained operation. Processing 20 items concurrently would convert a controlled population into a burst of requests. Batch size 1 makes the execution sequence explicit and allows a wait between items.



The loop is therefore a rate-control mechanism. It also makes failures easier to localise: one job can fail without requiring the entire population to be interpreted as a single model transaction.

16.2 Upsert

Upsert means 'update if the record already exists; insert if it does not'. MatchIQ uses this because a job is a persistent business entity, not a disposable execution item.

Why upsert	Business value
Stable Job Key	Same vacancy remains the same record across runs
Repeated runs	Existing assessment can be updated rather than duplicated
State fields	Processing status can evolve from NOT RUN → SUCCESS / FAILED
Auditability	Historical fields can be retained or extended without changing identity
Token optimisation	Known jobs can potentially be skipped or handled differently in future runs

17. Google Sheets and Operational Visibility

The exported workflow contains an 'Append row in sheet' Google Sheets node connected from the master data upsert path. The configured document is named 'MatchIQ Job Intelligence' and the sheet is Sheet1 in the exported configuration.

Field group	Examples	Operational purpose
Identity	run_timestamp, job_key	Record identity and run lineage
Job context	country, location, role, company	Human-readable opportunity context
Discovery	search_cluster, search_query, date_posted, linkedin_url	Source and discovery provenance
Deterministic scoring	scraper_match_score, opportunity_gate_score, talent_shortage_score	Explain pre-AI prioritisation
Process state	opportunity_status, agent1_status, agent2_status, agent3_status	Separate execution truth from fit
AI output	agent3_overall_fit_score, agent3_fit_level, agent3_recommendation	Decision-facing assessment
Assessment body	agent3_full_assessment	Retain full structured assessment
Failure	failure_reason	Make technical failure explicit

The development record also documents the intended Google Sheets role as an operational review surface rather than the system of record. The persistent master data and raw artefacts remain important for provenance and troubleshooting.

18. Development Environment and Dependency Bootstrap

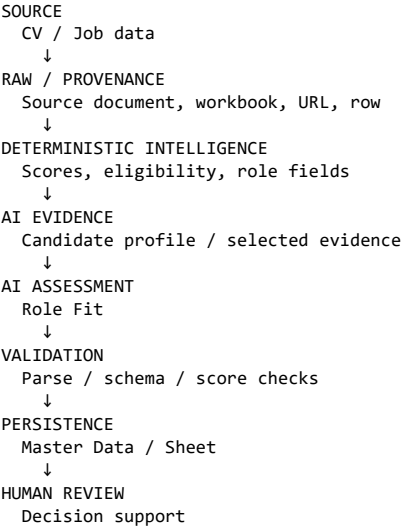
The project did not begin with a perfectly provisioned runtime. The dependency history is useful because it shows a diagnostic sequence: execute, observe the first missing dependency, install it, rerun, observe the next blocker, and continue until the script reaches its intended functional boundary.

Step	Observed event	Action	Purpose / learning
1	n8n failed with ModuleNotFoundError: requests	Verified requests dependency	HTTP acquisition required an explicit dependency
2	requests installed: 2.34.2	Installed package	Enable HTTP/HTTPS requests
3	LinkedIn request test	Confirmed connectivity	Separate network failure from application failure
4	HTTP 200 observed	Confirmed successful response	Prove endpoint reachability
5	pandas missing	Observed import failure	Identify data-processing dependency
6	pandas 3.0.5 installed	Installed package	DataFrames and Excel preparation
7	beautifulsoup4 4.15.0 missing	Installed package	HTML parsing
8	scikit-learn 1.9.1 missing	Installed package	TF-IDF / deterministic text similarity
9	numpy 2.5.3 installed as dependency	Dependency resolution	Numerical operations
10	scipy 1.18.1 installed as dependency	Dependency resolution	Scientific/numerical functions
11	joblib 1.6.0 installed as dependency	Dependency resolution	scikit-learn utilities
12	threadpoolctl 3.6.0 installed as dependency	Dependency resolution	Numerical thread control
13	openpyxl missing at Excel export	Installed package	XLSX writing
14	Windows CP1252 could not print emoji	Used python -X utf8	Prevent console encoding failure

This is a practical example of iterative troubleshooting rather than a polished-only narrative. The sequence demonstrates how environment failures were isolated by the next observable exception instead of being guessed in advance. (Source: MatchIQ development record.)

19. Output Architecture and Evidence Discipline

The system distinguishes between raw source, deterministic intelligence, semantic evidence, model output, validation and operational state. This prevents a technically failed assessment from being represented as a business decision.



State	Meaning	Must not be confused with
SUCCESS	The processing stage completed successfully	Good candidate
FAILED	Technical / processing failure	Poor candidate
NOT RUN	Stage intentionally not executed	No fit
EXCLUDED	Opportunity failed deterministic business criteria	AI rejection
DEFERRED	Opportunity retained but not processed now	Poor fit
Validated Role Fit Score	A valid semantic assessment was produced	Deterministic Match Score

The project framework explicitly requires that a failed or incomplete Agent 3 response never become a valid Role Fit Score; missing score means unvalidated processing, not zero fit. It also defines Capability Gap, Evidence Gap and Positioning Gap as distinct concepts. (Source: MatchIQ project framework.)

20. Data Sources, Lineage and Traceability

Layer	Source	Transformation	Persisted evidence
Candidate source	Google Docs / mutable CV document	Document extraction / CV versioning	candidate_id, cv_version, source content
Candidate semantic layer	Agent 2	Evidence-only profile generation	candidate_profile, updated_at
Job discovery	Python scraper / job-market source	Search, enrichment, normalisation	timestamped XLSX, source URL, source row
Job identity	LinkedIn URL or fallback key	Stable Job Key	job_key
Job intelligence	Python deterministic logic	Scoring, signals, constraints	Match Score, gate/market fields
Role intelligence	jd_intelligence.py	Structured role extraction	JSON role object
Candidate evidence selection	candidate_evidence_selector.py	Relevant evidence retrieval	selected_candidate_evidence
Role fit	Agent 3 / Groq HTTP development path	Semantic comparison	fit score, gaps, recommendation, summary
Operational persistence	Data Table / Google Sheets	Upsert / append	Master record + review surface

Lineage matters because the output is intended to support a real decision. A reviewer should be able to ask: where did this job come from, why was it selected, which candidate evidence was used, what did the model say, did validation succeed, and what is the current processing state?

21. Data Protection and Governance

The project distinguishes current MVP controls from production governance targets. This is important because the supplied PRD-style material contains proposed EEA-only architecture, confidential computing and zero-data-retention requirements that are not evidenced as current MatchIQ implementation.

Control	Current MVP evidence	Production target / limitation
Data minimisation	Agent 2 candidate evidence is separated from job processing; Agent 3 receives selected evidence	Formal data classification and minimisation policy
Purpose limitation	Candidate-role assessment is the explicit utility	Formal purpose register
Provenance	Stable Job Key, source URL, source workbook and run artefacts	Formal immutable audit trail
Failure separation	Failure Reason and status fields are separate from fit score	Central operational telemetry
Human oversight	System framed as decision support	Mandatory approval before consequential action
Inference boundary	Groq external endpoint used in MVP development	Approved EEA-controlled provider and contractual safeguards
Storage	XLSX, Data Tables, Google Sheets and raw files in MVP	Governed data store, retention and access controls
Residency	Not strict EEA-only in current MVP	Strict EEA boundary if required by deployment

The supplied MatchIQ framework explicitly states that the current MVP should not be represented as strict zero-EEA data egress and identifies EEA processing, EU infrastructure, retention, access and human approval as production requirements. (Source: MatchIQ project framework.)

The separate Agentic NLP presentation-generation PRD supplied in the project files contains a proposed enterprise architecture based on EEA sovereignty, local inference, Qdrant, confidential computing and an EU cloud boundary. Those are useful future-state reference concepts, but they are not treated here as implemented MatchIQ controls because they are not evidenced in the current MatchIQ workflow.

22. Hard Constraints and Control Logic

Constraint	Logic	Why hard / why deterministic
Role family	Included and not explicitly excluded	Prevents broad keyword similarity from admitting the wrong role family
Explicit exclusion	Always fails	Business rule must override semantic enthusiasm
Location / mobility	Configured scope	Practical eligibility cannot be inferred by LLM
Domain	Configured target domains	Preserve market relevance
Recency	Configured validity window	Avoid stale opportunities
Experience	Normalise explicit wording; do not equate literal phrase with candidate suitability	Prevents the documented 8–10-year logic failure
Duplicate	Stable Job Key	Prevents repeat opportunity processing
AI population	20 in current exported workflow	Controls model-facing workload
Serial execution	Loop Over Items	Controls provider pressure
Pacing	Wait node; 60 seconds in current export	Spreads calls
Role Fit validity	Parseable 0–100 score / schema output	Prevents invalid model output becoming a business score
Human decision	AI is decision support	Preserves accountability

23. Testing and Validation Evidence

Testing is described here as process validation rather than as a collection of technical test jargon. The project used failure observation, controlled execution, boundary testing, cache testing and output validation.

Test / observation	Recorded result	What it proved
LinkedIn connectivity	HTTP 200	Network path worked at the diagnostic point
1,313-job run	1,313 discovered; 5 selected	Experience logic was over-restrictive and market representation was suspect
Request volume	1,470 requests with repeated 429s	Enrichment strategy created an operational bottleneck
Opportunity Gate	1/60 passed ≥40	Additional downstream gate was too restrictive/redundant after upstream screening
Historical 20-job controlled run	20/20 AI executions, 20/20 Groq executions, 20/20 waits; no visible rate-limit failure	Serialisation/pacing architecture worked under the tested workload
CV versioning	Non-empty version identifier	Version generation worked
Cache first run	Stored version null → FALSE	Cache miss correctly routed to CV Intelligence
Cache later run	TRUE branch observed	Unchanged CV can bypass repeated inference
Agent 3 sample	66 / Partial fit / Proceed with targeted validation / failure_reason null	Validated assessment reached persistence
Agent 3 structured-output failure	Missing executive_summary under 2,200 completion-token setting	Output contract needed stronger capacity/instruction control
Google Sheets path	Connected from master-data output path	Operational review surface was architected

The project records distinguish completed observations from targets. For example, the 20 → 10 deterministic evidence filter is described as a target, while the 20-job controlled execution and cache behaviour are recorded observations. This distinction should remain in the final case study.

23.1 Testing methods used

- Dependency-by-dependency rerun testing: each missing package was identified by the next observable runtime exception.
- Connectivity testing: a direct HTTP 200 response separated network reachability from scraper logic.
- Boundary testing: 60-job population, 20-job AI budget and provider token constraints were treated as explicit limits.
- Negative-path testing: malformed/empty model responses, obsolete node references and parser failures were encountered and addressed.
- Cache testing: first-run cache miss and later cache hit were observed as different branches.
- Controlled-volume testing: a 20-job serialised run was used to validate pacing and provider behaviour.
- Output validation: Agent 3 was required to produce a parseable structured result before a fit score was treated as valid.

24. Lean Six Sigma Lens

The supplied project material does not claim a formal certified Lean Six Sigma project. The most defensible presentation is therefore to use DMAIC as a process-improvement lens applied retrospectively to the engineering work.

DMAIC phase	MatchIQ evidence	Capability demonstrated
Define	Reduce noise and uncertainty between job discovery and decision	Problem framing
Measure	1,313 discovered → 5 selected; 1,470 requests; 60 → 20 budget; 20/20 controlled run	Evidence-led measurement
Analyse	Experience logic, 429s, duplicated LLM work, repeated CV inference, parser failures	Root-cause analysis
Improve	Eligibility correction, deterministic JD logic, CV cache, serial loop, wait, evidence selector, Agent 3 transport redesign	Waste/failure reduction
Control	Stable Job Key, versioning, statuses, upsert, validation, raw artefacts and change-management framework	Sustained control

24.1 Waste categories visible in the project

Waste / defect type	MatchIQ example	Response
Over-processing	Full JD enrichment and AI treatment of too many opportunities	Deterministic filtering + AI budget
Waiting / queue pressure	Provider rate limits / 429s	Serialisation + wait
Defects	Malformed structured model response	Schema / parser validation
Rework	Repeated CV inference	Versioned cache
Duplication	Same job encountered through multiple city searches	Country-level final targeting + stable Job Key
Unnecessary motion	Multiple file handoffs	Future consolidation opportunity

25. Change Management and Decision Log

Change ID	Area	Observed problem	Decision / change	Value
CHG-001	Product logic	Opportunity relevance and candidate fit could be conflated	Separate opportunity relevance from candidate-role fit	Clearer decision architecture
CHG-002	Agent output	Structured/Pydantic output fragile	Controlled parsing / validation; later strict Groq HTTP path	More reliable output
CHG-003	JD intelligence	Generic governance template risk	Dynamic role-specific intelligence	Better hybrid-role relevance
CHG-004	CV intelligence	CV needed to be independent and reusable	Standalone evidence profile + cache	Reuse
CHG-005	Provider	Original provider limits constrained repeated calls	OpenAI-compatible integration + Groq	Provider flexibility
CHG-006	Tracing	Non-essential tracing produced 401	Disable non-critical tracing in MVP	Remove unnecessary failure path
CHG-007	Reliability	Empty/failed model responses	Bounded retries / backoff and explicit failure handling	Resilience
CHG-008	Discovery	City-level quotas caused duplicates	Country-level final targeting with city search anchors	Lower downstream volume
CHG-009	Token control	Too many jobs entering model path	Deterministic pre-screening + AI budget + context control	Predictable consumption
CHG-010	Data truth	Failed assessments could resemble valid results	Explicit statuses + no score on invalid output	Auditability
CHG-011	Identity	Repeated runs could duplicate jobs	Stable Job Key + upsert	Persistent entity identity
CHG-012	Opportunity Gate	Only 1/60 passed secondary threshold	Remove redundant rejection; retain score as data	Recall preservation
CHG-013	Agent architecture	Three LLM responsibilities	Retire JD LLM; retain Agent 2 + Agent 3	Lower unnecessary token use
CHG-014	Agent 3 transport	Missing executive_summary / structured-output failure	Increase completion budget and explicitly require all fields in development-record Groq HTTP path	Output contract reliability
CHG-015	Source document	Static PDF source awkward for changes	Move toward mutable Word source	Source-level change management
CHG-017	Ingestion / persistence architecture	Google Sheet was reported as not updating; main.py had to become the controlled entry point for the new job.py → Job Intake architecture.	main.py invokes job.py; Google Sheets Job Intake is the persistent destination; XLSX export is disabled; post-run row-count/latest-record verification is required.	Stable persistence boundary, reduced file dependency, observable ingestion and cleaner hand-off to n8n.

25.1 Formal change-management method

1. Record the problem or business requirement.
2. Identify the affected component and version.
3. Describe old behaviour and proposed new behaviour.
4. Assess impact on quality, cost, latency, reliability, output schema, privacy and downstream decisions.
5. Test against a fixed benchmark or controlled execution.
6. Approve according to risk.
7. Increment the relevant component version.
8. Deploy with rollback capability.
9. Observe post-change behaviour and record follow-up actions.

25.2 Latest Workflow Correction — 16 September 2026

The latest development cycle did not add more agents or more orchestration. It reduced the workflow to the components already agreed for the current MatchIQ design and then traced a downstream failure back to its actual data-source dependency. This is important because the failure initially appeared to be a JavaScript/context problem, but the evidence showed that the upstream job retrieval node was returning an empty object.

Observed failure: Get Jobs for Agent 3 was configured as an HTTP GET to the local webhook endpoint /webhook/matchiq/jobs. The node returned an item with an empty JSON object rather than a populated job record. Prepare Agent 3 Context therefore had no reliable job_key, role or job intelligence to pass downstream. The resulting job_key error was a symptom of missing upstream data, not evidence that the context-preparation JavaScript itself was the primary fault.

Design correction: the Agent 3 branch was redefined so that job data and candidate evidence have separate responsibilities. A job-reading node is responsible only for supplying job records; a candidate-profile read is responsible only for supplying the reusable Candidate Evidence Profile. This removes the earlier ambiguity in which Get row(s) was being used conceptually for both candidate evidence and job input.

The intended corrected branch is: Get Jobs for Agent 3 → Prepare Agent 3 Context → Agent 3 - Groq HTTP → Parse Role Fit → Upsert rows1 → Append row in sheet. The candidate branch remains responsible for CV retrieval, CV versioning/cache logic, Agent 2 and candidate-profile persistence. The HTTP webhook dependency and the obsolete Top-20 budget node are not part of the intended simplified Agent 3 path.

Why this change was made: the objective was not merely to remove nodes. It was to restore a clean data lineage in which each node has one business purpose, the source of each input is explicit, and downstream code does not compensate for an upstream source-selection error. This is consistent with the project's broader control principle: diagnose the failure at the correct process boundary before changing the semantic decision layer.

Implementation status: the correction has been specified from the observed workflow behaviour, but the supplied development record ends at the point where the replacement Google Sheets Get Row(s) node was to be created and configured. It should therefore be treated as an approved workflow correction pending execution and verification, not as a completed production change.

The JavaScript correction itself was also narrowed. Rather than continually expanding defensive parsing logic, the design now expects a valid job record at the input boundary and uses explicit validation for job_key and role. This avoids masking a broken upstream source with increasingly complex downstream fallbacks.

What we observed / changed	Why	Control value
CHG-016 — Workflow orchestration Reduced the workflow to the agreed active components.	Prevent retired architecture from being reintroduced during debugging.	Architecture discipline
Identified Get Jobs for Agent 3 as an HTTP webhook returning an empty object.	The downstream job_key error originated from missing upstream job data.	Root-cause diagnosis
Separated job retrieval from candidate-evidence retrieval.	One node should not carry two different data responsibilities.	Clear data lineage
Defined Google Sheets Get Row(s) as the direct job source for the Agent 3 branch.	Supply populated job records without the failing local webhook dependency.	Reliable input boundary
Removed the need for downstream	Fix the process boundary rather than	Maintainability

JavaScript to compensate for a bad source.	adding parser complexity.	
Kept Agent 3 unchanged at the semantic decision level.	The defect was in orchestration/data flow, not evidence-based role-fit logic.	Change isolation

25.3 Latest Ingestion Architecture Correction — 16 September 2026

A further correction was made after the Google Sheet appeared not to update. The objective was to establish one authoritative ingestion path and make persistence observable rather than relying on an exported workbook as an intermediate source.

The implemented entry-point design is now: main.py → job.py → Google Sheets / Job Intake → n8n. The previous main.py pattern, which was centred on new.py → XLSX → downstream agent processing, was obsolete for the reduced architecture. The revised main.py calls job.py as the production ingestion engine and identifies Google Sheets, specifically the Job Intake tab, as the persistent destination. XLSX export is disabled in this path.

The orchestration layer was deliberately kept separate from the ingestion engine. job.py performs job discovery, parsing, enrichment, qualification and persistent deduplication; Google Sheets / Job Intake acts as the durable job-intake layer; n8n subsequently consumes the persisted job population for downstream processing. This establishes a clearer system boundary: discovery and persistence occur before AI assessment.

Evidence from the run: before execution, Job Intake contained 812 data rows. The runtime banner reported the production engine as job.py, the Google Sheet tab as Job Intake, and persistent deduplication as being handled by job.py. The scraper then began country-level discovery across Luxembourg, Germany, Italy, Spain, Switzerland and Poland using the configured search modes. These observations establish that the new main.py was actually invoking the intended ingestion architecture.

A separate operational issue occurred when an attempted Copy-Item command referenced main_updated.py, which did not exist in the local project directory. That error was not a pipeline defect: the subsequent direct execution of `.\venv\Scripts\python.exe .\main.py` demonstrated that the existing main.py was already using the new architecture. The correction therefore avoided unnecessary file replacement and moved to runtime verification.

The normal operating command is now intentionally simple: `.\venv\Scripts\python.exe .\main.py`. job.py should not normally be run separately because main.py is the controlled entry point. After a run, persistence is verified independently by reading Job Intake and checking its data-row count and latest record. This creates an explicit control between process completion and data persistence.

Why this change matters: the project moved from a file-export workflow towards a persistent intake architecture. This reduces dependence on transient XLSX artefacts, gives n8n a stable source boundary, preserves deduplication at ingestion, and makes it possible to verify whether a completed scraper run actually produced durable job records. It also supports the user's original requirement that one run writes a new batch below previous batches rather than repeatedly producing duplicate files.

The Google integrations have distinct functional roles. Google Docs provides the live candidate source document so the candidate evidence profile can be refreshed from the current CV rather than from a manually copied text block. Google Sheets provides persistent operational storage for Job Intake and the downstream MatchIQ Job Intelligence output. Separating source-document management from job-intake persistence supports source-level change: a CV change can update candidate evidence, while a new job run can append only newly identified jobs.

The latest change also reinforces the Opportunity Gate design decision. Opportunity Gate remains a discovery/prioritisation signal rather than an additional downstream rejection rule. The earlier experiment in which an Opportunity Gate ≥40 filter allowed only 1 of 60 selected jobs to pass demonstrated that a technically valid control could be operationally redundant when the upstream scraper had already performed opportunity selection. The score was therefore retained as evidence and prioritisation data while the redundant downstream rejection step was removed.

Current evidence boundary: the supplied run log proves that main.py invoked job.py and that Job Intake contained 812 rows before the run. The supplied record does not yet provide the final post-run row count for this particular execution. Therefore the document records the ingestion architecture as implemented and the persistence verification as the required control, but does not claim a new batch size that has not been evidenced.

Component	Functional role	Technical role	Why it was added / retained
main.py	Single controlled entry point for the current pipeline.	Invokes job.py and exposes the configured Google Sheet destination.	Prevents operators from running separate ingestion components inconsistently.

job.py	Discovers and qualifies the target job population.	Search, parsing, enrichment, scoring/qualification and persistent deduplication.	Moves deterministic ingestion and persistence into one controlled engine.
Google Sheets — Job Intake	Persistent job-intake population for downstream processing.	Stores rows across runs rather than creating a new workbook for each run.	Supports batch accumulation, deduplication and observable persistence.
n8n	Operational orchestration and downstream AI workflow.	Consumes persisted job records and coordinates candidate/role-fit processing.	Separates operational automation from deterministic discovery.
Google Docs	Live candidate source.	Supplies the current CV content to Candidate Intelligence.	Allows source-level CV changes to flow through the matching process.
Persistence verification	Confirms that ingestion produced durable records.	Reads Job Intake row count and latest record after execution.	Distinguishes 'script completed' from 'business data persisted'.

26. Versioning Model

Version dimension	What changes	Why independently version it
PROJECT	Overall MatchIQ release	Provides release identity
SCRAPER	Discovery, enrichment, taxonomy and quota logic	Separates market acquisition changes
SCORING	Professional DNA weights, signals and gate logic	Allows score calibration without changing agents
AGENT	Prompts and orchestration logic	Controls semantic behaviour
MODEL	Provider, model and inference configuration	Controls provider/model changes
OUTPUT	Spreadsheet schema, statuses and persistence	Protects downstream consumers
INFRASTRUCTURE	Runtime, packages and deployment	Makes environment changes reproducible

The project framework gives an example release structure such as MatchIQ 1.1 with separate scraper, scoring, agents, model and output identifiers. The principle is more important than the exact version number: a prompt change should not be hidden inside a general project-version increment.

27. Novelty and Differentiation

The project is differentiated less by the presence of an LLM than by the architecture around it. Many AI demonstrations start with a document and end with a generated answer. MatchIQ starts with a decision problem and uses AI only where deterministic processing stops being sufficient.

Differentiator	Evidence in MatchIQ	Why it is sophisticated
Decision separation	Opportunity relevance vs candidate-role fit	Prevents one score from carrying incompatible meanings
Selective AI	60 → 20 budget; deterministic processing before LLM	Treats inference as a scarce resource
AI removal as optimisation	Agent 1 retired; JD intelligence moved to Python	Shows architecture chosen for process value, not AI novelty
Reusable candidate intelligence	Agent 2 + CV version/cache	Turns a CV into a reusable information product
Hybrid-role mapping	Professional DNA + functional signal families	Reduces title-only matching bias
Evidence discipline	Explicit evidence-only instructions; gaps separated from absence	Improves explainability
Failure truth	FAILED / NOT RUN / EXCLUDED / validated score separated	Prevents operational defects becoming business conclusions

Differentiator	Evidence in MatchIQ	Why it is sophisticated
Persistent identity	Stable Job Key + upsert	Creates repeatable entities across runs
Low-code + code division	n8n orchestration; Python deterministic computation	Uses each mechanism where it is strongest
Change-at-source model	Mutable CV/source + versioning	Supports maintainable role-specific operation

The novelty can therefore be expressed as controlled orchestration of heterogeneous decision mechanisms: deterministic rules for what can be known reliably, AI for what needs contextual judgement, and workflow controls for how expensive and consequential operations are executed.

28. Business Value and Transformation Capability

Business value	Mechanism	Evidence / metric
Reduced review burden	Prioritisation before semantic assessment	60 → 20 current AI budget
Reduced repeated AI work	Candidate evidence cache	Cache hit branch observed
Improved consistency	Deterministic eligibility and scoring	Explicit Professional DNA and hard constraints
Better decision transparency	Evidence, gaps, recommendation and status fields	Agent 3 output contract
Improved reliability	Serialisation, wait, parser and validation	20/20 controlled execution evidence
Improved maintainability	Separate scripts, parsers, source versioning	Explicit interfaces
Scalability	Bounded model-facing population + reusable candidate layer	Architecture supports controlled expansion
Auditability	Stable Job Key, provenance, raw artefacts and upsert	Persistent master dataset

No application-conversion, interview-conversion or time-saved figures are claimed because those outcome measurements were not supplied. The project demonstrates operational and architectural value without inventing business results.

29. Scalability: What Changes When Volume Increases

Scaling dimension	Current design	Controlled next step
Jobs	60 selected; 20 AI budget	Benchmark larger populations after telemetry
Candidates	One candidate cache model	Extend cache by candidate/tenant identity
LLM calls	Two active semantic components; one per-job Role Fit	Measure token/job and validated assessment cost
Deterministic work	Python scripts with file handoffs	Consolidate where regression proves safe
Persistence	Data Tables + XLSX + Google Sheets	Governed database + dashboard
Provider capacity	Serial + wait	Only introduce concurrency after capacity measurement
Quality	Manual / controlled validation	Fixed benchmark suite and human-agreement metrics
Governance	MVP controls	Formal retention, residency, access and approval

The central scalability principle is not concurrency first. It is reduction first: reduce unnecessary data, unnecessary calls, unnecessary repeated work and unnecessary state transitions before adding throughput.

30. KPI and Measurement Framework

KPI family	Measure	Why it matters
Discovery	Jobs collected, unique jobs, country coverage, full-JD enrichment rate	Measures source quality
Screening	Gate-pass rate, exclusion rate, deduplication rate	Measures deterministic funnel behaviour
AI economics	Tokens/job, cost/job, cost/validated assessment, retry cost	Measures resource efficiency

KPI family	Measure	Why it matters
AI quality	Agent completion, validated-score rate, human agreement	Measures semantic reliability
Reliability	429 rate, provider failure rate, parse failure rate, write failure rate	Measures operational health
Latency	Time from discovery to persisted assessment	Measures user experience
Decision value	Actionable-opportunity rate, application/interview conversion when available	Measures actual business impact
Governance	Provenance completeness, human-review completion	Measures accountability

31. Current MVP vs Target Production

Area	Current evidenced state	Target state
Orchestration	n8n 2.38.7	Production-controlled orchestration with source control
Runtime	Local Windows environment	Controlled enterprise runtime
Inference	Groq external endpoint in development/current path	Approved provider with residency/security controls
Candidate source	Google Docs	Governed mutable source
Job persistence	XLSX + Data Table + Google Sheets	Governed database + dashboard
JD intelligence	Deterministic Python	Versioned service/script with regression suite
Candidate evidence	Agent 2 + cache	Multi-candidate governed cache
Role Fit	Agent 3; strict Groq HTTP development path	Approved model/provider with benchmarked contract
Human review	Decision-support design	Mandatory approval for consequential action
Residency	Not strict EEA-only	Strict EEA boundary where required
Observability	Console + run artefacts	Central telemetry for quality/cost/reliability

32. Open Items and Evidence Boundaries

Open item	Why it remains open	Correct treatment in case study
Complete current jd_intelligence.py source	Not included in supplied workflow export	Describe interface/function; do not fabricate source
Complete final Agent 2 prompt archive	Current workflow prompt exists; development history may contain revisions	Use current exported prompt and label other revisions as history
Complete final Agent 3 Groq HTTP JSON body	Development record contains it, but exported workflow contains a native Agent 3	Describe both with evidence labels
Exact token savings	Population reductions are known; token measurements are not	Do not convert 60 → 20 into claimed token savings
20 → 10 evidence filter	Discussed as target	Do not call completed without run evidence
Production EEA controls	MVP uses external Groq and local/Google/XLSX components	Present as target, not current compliance
Full test count	Execution history does not provide a complete single count	Use named recorded tests
Google Cloud Project configuration	Not evidenced as a distinct implemented project in supplied workflow	Do not claim it as current
Antigravity optimisation completion	Audit/optimisation work was interrupted by quota in the development record	Present as engineering/QA concept, not completed deployment

This section is intentionally conservative. The strength of the case study is improved, not weakened, by distinguishing implemented behaviour from planned behaviour and by refusing to fill evidence gaps with plausible-sounding technical detail.

33. Appendix A — Prompt Engineering and Agent Modification Logic

Prompt engineering in MatchIQ was used as a process-control mechanism. The development record describes using ChatGPT first as a data scientist/process engineer to preserve context and reduce hallucination, then as a professional project-document drafter while maintaining change-management logic. The important point is that prompting was used to preserve system boundaries, not merely to make prose sound better.

Prompt stage	Control objective	Observed design
Role instruction	Tell the model what responsibility it owns	Agent-specific system/task framing
Evidence discipline	Prevent invented capability	Use only explicit candidate evidence
Separation of concerns	Prevent cross-contamination	Agent 2 cannot see jobs; Agent 1 originally could not see CV
Output contract	Make downstream data predictable	Fixed headings / JSON schema
Token discipline	Reduce unnecessary context	Compact profile, selected evidence, explicit output limits
Change control	Make prompt changes reviewable	Version / change-log principle

33.1 Current Agent 2 prompt — full recorded text

=Analyze the following CV for MatchIQ.

CV TEXT:

```
{{ $('Get a document').item.json.content }}
```

Create a compact reusable Candidate Evidence Profile for later role-fit assessment.

Use ONLY information explicitly stated in the CV. Never infer or invent capabilities, technologies, responsibilities, authority, qualifications, certifications, budget ownership, people management, or technical experience.

Do not assess fit for any job.

Do not score the candidate.

Do not recommend jobs.

Extract only high-value, non-duplicated evidence under these headings:

PROFILE

2-3 concise lines covering career progression, professional focus, and seniority.

CORE EXPERTISE

6-10 strongest explicitly evidenced functional capabilities.

CAREER EVIDENCE

8-12 strongest roles, responsibilities, programmes, and projects. Prioritize ownership, scope, and distinctive experience.

DOMAIN EVIDENCE

5-8 evidenced industries, business domains, regulatory environments, or organisational contexts.

LEADERSHIP & STAKEHOLDERS

5-8 evidenced examples of leadership, programme/product ownership, cross-functional delivery, senior stakeholder engagement, international delivery, or change leadership. Distinguish leading, owning, coordinating, contributing, and participating.

TECHNICAL & DIGITAL

8-12 explicitly evidenced technologies, platforms, AI/data capabilities, automation tools, and methodologies.

GOVERNANCE & REGULATORY

5-8 explicit examples of AI governance, risk, compliance, controls, assurance, regulatory frameworks, or operating models.

QUANTIFIED ACHIEVEMENTS

5-8 strongest measurable outcomes. Preserve the original figures.

SENIORITY

2-3 concise lines describing the level of responsibility explicitly demonstrated.

EVIDENCE GAPS

Up to 5 important areas where the CV provides limited or no explicit evidence. Do not treat absence of evidence as proof that the candidate lacks the capability.

Rules:

- Prefer evidence over narrative.
- Preserve exact numbers, role titles, industries, technologies, frameworks, and certifications.
- Remove duplicates.
- Do not convert participation into ownership.
- Do not convert a team size into people management.
- Do not infer cloud, cybersecurity, security architecture, MLOps, hands-on engineering, or budget ownership unless explicitly stated.
- If evidence is ambiguous, omit it.
- Return plain text.
- Use exactly the ten headings above.
- Keep the complete profile concise, ideally under 1,000 tokens.

33.2 Original Agent 1 prompt — historical / retired

=Analyze this job for MatchIQ.

ROLE:

{{ \$json.role }}

COMPANY:

{{ \$json.company }}

LOCATION:

{{ \$json.location }}

FULL JOB DESCRIPTION:

{{ \$json.full_job_description }}

Produce a compact, evidence-based Job Intelligence summary for downstream candidate assessment.

Identify only what is supported by the job description:

1. ROLE PURPOSE

One concise statement.

2. TOP RESPONSIBILITIES

List the 5 most important responsibilities.

3. REQUIRED CAPABILITIES

List the 5 most important capabilities, skills, or competencies.

4. SENIORITY AND LEADERSHIP

State the expected seniority, ownership, leadership, and stakeholder expectations.

5. SUCCESS OUTCOMES

List the 3 most important outcomes expected from the person hired.

6. ASSESSMENT CRITERIA

List the 5 most important criteria that should be used later to assess candidate fit.

Do not assess any candidate.

Do not use or infer information from a CV.

Do not invent requirements.

Do not repeat the job description unnecessarily.

Be concise and prioritize decision-useful information.

Return exactly these sections:

ROLE PURPOSE

TOP RESPONSIBILITIES

REQUIRED CAPABILITIES

SENIORITY AND LEADERSHIP

SUCCESS OUTCOMES

ASSESSMENT CRITERIA

Agent 1 is included because it explains the architecture's evolution. It is not presented as an active final component. The exported workflow connections show the Agent 1 main path disconnected. (Source: MatchIQ project framework.)

33.3 Current exported native Agent 3 prompt — full recorded text

=Assess the candidate's fit for this specific role using ONLY the Job Intelligence and Selected Candidate Evidence below.

JOB INTELLIGENCE

Role:

{{ \$json.role }}

Company:

{{ \$json.company }}

Location:

{{ \$json.location }}

Role Purpose:

{{ \$json.jd_role_purpose }}

Top Responsibilities:

{{ \$json.jd_top_responsibilities.join('\n- ') }}

Required Capabilities:

{{ \$json.jd_required_capabilities.join('\n- ') }}

Seniority and Leadership:

{{ \$json.jd_seniority_leadership }}

Assessment Criteria:

{{ \$json.jd_assessment_criteria.join('\n- ') }}

SELECTED CANDIDATE EVIDENCE

{{ \$json.selected_candidate_evidence }}

DETERMINISTIC EVIDENCE SIGNALS

Matched Job Signals:

{{ \$json.matched_job_signals.join(', ') }}

Deterministic Evidence Score:

```
{{ $json.deterministic_evidence_score }}
```

Deterministic Signal Coverage:

```
{{ $json.deterministic_signal_coverage }}%
```

ASSESSMENT RULES

Assess the candidate against the actual requirements of this specific role.

Use only evidence explicitly supported by the Selected Candidate Evidence.

Distinguish clearly between:

- demonstrated evidence
- transferable experience
- evidence gaps
- genuine risks

Do not treat an absence of CV evidence as proof that the candidate lacks a capability.

Do not invent experience, technologies, certifications, responsibilities, authority, or qualifications.

Do not infer cloud, cybersecurity, security architecture, MLOps, threat modelling, incident response, or hands-on engineering experience unless explicitly evidenced.

Evaluate both direct fit and transferable fit.

Be balanced. Do not over-penalize transferable experience, but do not treat transferable experience as equivalent to direct hands-on experience.

For certifications, distinguish between security certifications required by the role and other certifications actually present in the candidate evidence.

The deterministic evidence score and signal coverage are retrieval indicators only. Do not treat them as the candidate's final fit score.

OUTPUT RULES

Return only the structured output required by the Output Parser.

Do not use Markdown.

Do not use **, *, #, headings with Markdown formatting, or Markdown code fences.

Keep every text value concise and professional.

Do not repeat the same evidence across multiple fields.

SCORING

overall_fit_score:

0-39 = Poor fit
40-59 = Limited fit
60-74 = Partial fit
75-89 = Good fit
90-100 = Excellent fit

The score must reflect the actual evidence and importance of the role requirements.

fit_level must correspond to the score range.

recommendation must be one of:

- Proceed to human review
- Proceed with targeted validation
- Do not prioritize

strongest_fit_evidence:

Provide exactly 3 concise evidence-based points.

key_gaps_and_risks:

Provide exactly 3 important gaps or risks relevant to this specific role.

transferable_experience:

Provide exactly 2 areas where existing experience can reasonably transfer to this role.

evidence_gaps:

Provide up to 3 important areas where the CV provides limited or no explicit evidence.

assessment_summary:

Provide one concise paragraph of approximately 60-100 words explaining the overall assessment, balancing direct evidence, transferable experience, and evidence gaps.

33.4 Development-record Agent 3 Groq HTTP prompt controls

System role:

You are Agent 3 for MatchIQ. You are an experienced HR professional and Hiring Manager assessing one specific job against one specific candidate.

Core controls:

- deterministic MatchIQ layer has already performed broad job matching;
- do not redo keyword matching;
- use only supplied job information and candidate evidence;
- do not invent skills, tools, languages, domains, responsibilities, certifications, leadership scope or experience;
- distinguish PRESENT, PARTIAL, MISSING and NOT_EVIDENCED;
- distinguish direct experience, transferable experience, participation, ownership and technical exposure;
- avoid repeated evidence;
- prioritise material hiring implications;
- return valid JSON only.

Required output:

overall_fit_score
fit_level
recommendation
overall_hiring_assessment
skills_tools_mapping
language_mapping
domain_mapping

```
functional_responsibility_mapping
transferable_experience
gaps_risks_evidence_gaps
hiring_manager_view
executive_summary
```

Executive summary:
80–120 words and final field.

```
Inference configuration recorded:
model = openai/gpt-oss-120b
temperature = 0.2
reasoning_effort = low
max_completion_tokens = 3500
response_format = json_schema
strict = true
```

34. Appendix B — Coding and Configuration Evidence

The following items are exact configuration evidence from the exported workflow, rather than reconstructed source code.

Component	Recorded configuration
n8n	2.38.7 recorded in the project architecture documentation
Python command for job engine	python -X utf8 "C:\Users\Susmita\OneDrive\Desktop\Match IQ\new.py"
Python CV extraction command	python -c "from docx import Document; d=Document(r'C:\Users\Susmita\OneDrive\Desktop\Match IQ\Resume\Susmita Kaya.docx'); parts=[p.text.strip() for p in d.paragraphs if p.text.strip()]; parts += [' '.join(c.text.strip() for c in row.cells if c.text.strip()) for t in d.tables for row in t.rows if any(c.text.strip() for c in row.cells)]; print('\n'.join(parts))"
JD script command	python "C:\Users\Susmita\OneDrive\Desktop\Match IQ\jd_intelligence.py" < "C:\Users\Susmita\OneDrive\Desktop\Match IQ\matchiq_job.json"
Candidate selector command	python "C:\Users\Susmita\OneDrive\Desktop\Match IQ\candidate_evidence_selector.py" < "C:\Users\Susmita\OneDrive\Desktop\Match IQ\matchiq_candidate_evidence.json"
Job population Limit	maxItems = 20
Sort	Match Score descending
Wait	60 seconds in exported workflow
Agent 2 model	openai/gpt-oss-120b
Agent 2 maxTokensToSample	1100 in exported native model configuration
Agent 2 temperature	0.3 in exported native model configuration
Native Agent 3 model	openai/gpt-oss-120b
Native Agent 3 maxTokensToSample	1400 in exported native model configuration
Native Agent 3 temperature	0.7 in exported native model configuration
Google Sheet document	MatchIQ Job Intelligence
Google Sheet tab	Sheet1
Candidate Data Table	MatchIQ_Candidate_Profile
Master Data Table	MatchIQ_Master_Job

The development record separately documents the later Agent 3 Groq HTTP settings of 0.2 temperature, low reasoning effort and 3,500 completion tokens. This difference is precisely why versioned configuration matters: an exported workflow and a later working-development path can otherwise be confused.

34.1 Known deterministic interfaces

JD Intelligence interface

INPUT:
matchiq_job.json

PROCESS:
jd_intelligence.py

OUTPUT:
JSON via stdout

VALIDATION:
Parse JD Intelligence

Candidate Evidence Selector interface
INPUT:

matchiq_candidate_evidence.json

PROCESS:
candidate_evidence_selector.py

OUTPUT:
JSON via stdout

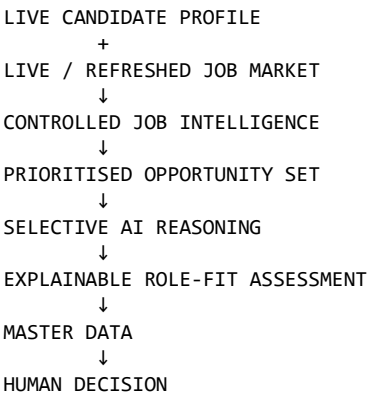
VALIDATION:
Parse Candidate Evidence

35. Appendix C — Master Data Field Model

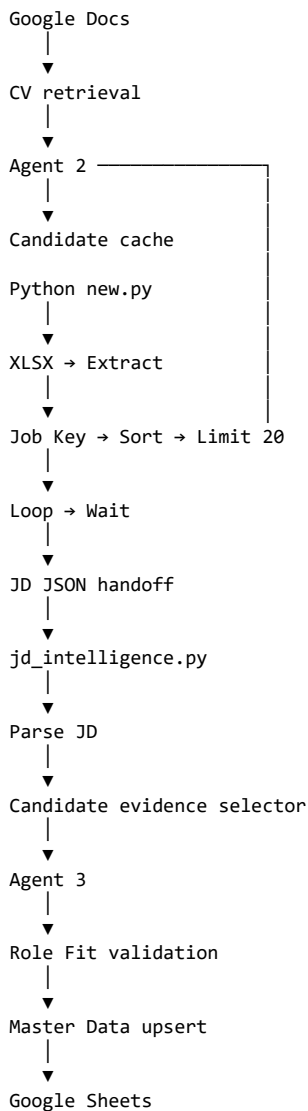
Field	Role in system	Data type / meaning
run_timestamp	Execution lineage	Timestamp
job_key	Stable entity identity	String
country	Geographic context	String
location	Role location	String
role	Job title / role label	String
company	Employer	String
search_cluster	Discovery rationale	String
search_query	Discovery provenance	String
date_posted	Recency	String/date
linkedin_url	Source identity	String
scraper_match_score	Deterministic relevance	Numeric
opportunity_gate_score	Opportunity signal	Numeric
talent_shortage_score	Market signal	Numeric
opportunity_status	Business/processing state	String
agent1_status	Historical/role-intelligence state	String
agent1_role_analysis	Historical role output	String
candidate_id	Candidate identity	String
agent2_status	Candidate evidence processing state	String
agent3_status	Role-fit processing state	String
agent3_overall_fit_score	Validated role-fit score	Numeric
agent3_fit_level	Human-readable fit classification	String
agent3_recommendation	Action recommendation	String
agent3_full_assessment	Structured role-fit evidence	String/JSON serialisation
failure_reason	Technical failure explanation	String

36. Appendix D — Architecture Diagrams in Narrative Form

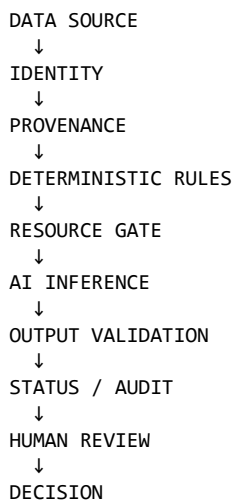
36.1 Business-level view



36.2 Technical-level view



36.3 Governance-level view



36.4 Stable n8n Workflow — Current Operating Design (17 September 2026)

The latest workflow inventory establishes the current stable design as two coordinated paths: a reusable candidate-evidence/cache path and a per-job assessment path. This section supersedes older diagrams where they conflict with the

current node inventory. It preserves the distinction between components already validated and the continuous-run controls that remain to be verified.

Candidate evidence path: When clicking 'Execute workflow' → Get a document → CV Version → Get Candidate Profile Cache → CV Version Cached? → Agent 2 - CV Intelligence (only when required) → Upsert row(s). Get row(s) then provides the stored candidate profile to the assessment path. The cache decision must validate both the CV version and that the cached candidate_profile is non-empty; a matching version alone is not sufficient.

Job assessment path: Manual Trigger → read candidate profile and Job Intake → Select Agent 3 Batch → Merge → Prepare Agent 3 Context → Loop Over Items (Batch Size 1) → Wait (approximately 40 seconds) → Agent 3 - Groq HTTP → Parse Role Fit → Upsert Rows1 → Google Sheets. The inventory reports that Job Intake returned 827 items in one execution; this is a snapshot from that execution, not a guaranteed current row count.

The batch selector is intended to select at most 20 rows where ingestion_status is NEW and agent3_status is not SUCCESS. This is a processing budget, not a claim that only 20 jobs exist. Loop Over Items with Batch Size 1 serialises individual calls; the approximately 40-second Wait is a provider-throughput control. The complete next-batch continuation is not yet verified, so the design should not be described as an unattended infinite queue.

The Merge node combines the candidate record and selected job records. A known failure appeared on a subsequent pass when the candidate profile was absent from the Merge input. The latest proposed Prepare Agent 3 Context code addresses this by first searching Merge items for a candidate-only record, then falling back to the already-executed Get row(s) candidate lookup, validating that candidate_profile is non-empty, identifying job items by job_key, and creating one context item per job. The supplied inventory explicitly marks this fallback as proposed and not yet confirmed in the live workflow.

Prepare Agent 3 Context creates a stable per-job contract: job_key, role, company, location, country, job_intelligence, candidate_id, candidate_evidence (including candidate_profile and cv_version), and an evidence_boundary that prohibits invented candidate claims. It validates that job_key and job records exist. The intention is to make data provenance and evidence constraints explicit before the model call.

Agent 3 - Groq HTTP uses the recorded Groq chat-completions endpoint, model openai/gpt-oss-120b, temperature 0.2, low reasoning effort, 3,500 maximum completion tokens and strict JSON Schema output. Parse Role Fit converts the response into structured assessment fields. Upsert Rows1 persists the job-level result in MatchIQ_Master_Job. The Google Sheets append node was shown configured for MatchIQ Job Intelligence → Sheet1; it must not be mistaken for the persistent Job Intake queue or for an update to the original queue row.

Critical queue-control distinction: writing a successful assessment to MatchIQ_Master_Job does not, by itself, update the original Job Intake row. For reliable continuation, the matching Job Intake row must be updated in place with agent3_status = SUCCESS and processed_at (or an equivalent controlled status). The inventory identifies this update as essential and not yet verified. An append-only write could create duplicates and leave the original job eligible for selection again.

Current validation boundary: CV retrieval, Agent 2 generation, candidate cache upsert/lookup, one successful Agent 3 context preparation, Agent 3 structured assessment and Master Job upsert have been observed as successful. The candidate fallback on subsequent loop passes, correct Job Intake source configuration, in-place queue status update and full next-batch continuation remain open verification items. Accordingly, the stable architecture is documented as the current design, but end-to-end continuous queue processing is not yet represented as fully validated.

Node / stage	Input → output	Business / technical purpose	Evidence status
Get a document	Google Docs CV → document content	Live candidate source; avoids manual CV text copying.	Validated: full CV content returned.
CV Version	CV/document → cv_version	Cache invalidation signal when the source CV changes.	Current node exists; exact implementation details depend on its live configuration.
Get Candidate Profile Cache	candidate_id/version criteria → cached profile record	Reuses prior Agent 2 output rather than reprocessing unchanged CV.	Lookup and populated profile observed.
CV Version Cached?	current version + cache state → true/false	Runs Agent 2 only when cache is stale or profile missing.	Decision must also test non-empty profile.

Agent 2 - CV Intelligence	CV text → reusable evidence profile	Extracts CV-grounded evidence without assessing a particular vacancy.	Successful generation observed; shorter profile remains an optimisation item.
Upsert row(s)	candidate_id, profile, cv_version, updated_at → cache record	Maintains one reusable candidate profile record.	Successful cache write observed.
Get row(s)	candidate Data Table → candidate record	Provides candidate evidence independently of job rows.	Populated profile observed.
Get row(s) in sheet	Job Intake → job rows (one item per row)	Reads persistent job queue for eligible assessment work.	827 items reported in one execution; confirm live spreadsheet/tab binding.
Select Agent 3 Batch	queue rows → up to 20 eligible jobs	Selects NEW rows not already SUCCESS; enforces model-call budget.	Code supplied; live behavior should be regression-tested.
Merge	candidate record + selected jobs → combined items	Joins shared candidate evidence to per-job records.	21 items observed for 1 candidate + 20 jobs; later pass exposed missing candidate item.
Prepare Agent 3 Context	merged items → one context per job	Validates job_key/profile and builds a bounded evidence contract.	Worked once; fallback for later passes proposed, not verified.
Loop Over Items	prepared contexts → one job at a time	Controls concurrency and enables per-item processing.	Batch Size 1 recorded.
Wait	job item → delayed job item	Reduces call rate and mitigates provider throttling.	Approximately 40 seconds recorded in current inventory.
Agent 3 - Groq HTTP	context → structured role-fit response	Assesses direct and transferable fit against job-specific criteria.	Successful structured output observed.
Parse Role Fit	HTTP response → parsed fields	Separates transport response handling from data validation/persistence.	Complete assessment parsed successfully.
Upsert Rows1	assessment + job metadata → MatchIQ_Master_Job	Updates/creates durable job assessment by stable key.	Successful record observed; score mapping correction recorded.
Append row in sheet	Master Job output → MatchIQ Job Intelligence / Sheet1	Provides a spreadsheet output view; not automatically a queue update.	Node shown; confirm purpose/operation and do not confuse with Job Intake status update.
Job Intake status update	job_key + status/time → original queue row updated	Prevents successful jobs from being reselected and supports safe continuation.	Essential control; not yet verified.

36.5 Current Stable Queue Logic — Supplied Selector

The supplied selector uses the following business rule: include a row only when ingestion_status is NEW and agent3_status is not SUCCESS; stop after 20 selected items. This is a bounded batch selector. It does not itself mark jobs as processed, fetch the next batch, or prove that the original Job Intake row is updated.

```

const rows = $input.all();
const selected = [];

for (const item of rows) {
  const row = item.json;
  const ingestionStatus = String(row.ingestion_status ?? "").trim().toUpperCase();
  const agent3Status = String(row.agent3_status ?? "").trim().toUpperCase();

  if (ingestionStatus === "NEW" && agent3Status !== "SUCCESS") {
    selected.push(item);
    if (selected.length >= 20) break;
  }
}
return selected;

```

36.6 Operational Acceptance Checks

Before describing the queue as continuously operational, verify in one controlled run: (1) the Job Intake reader points to the intended spreadsheet and tab; (2) the selector returns no more than 20 eligible jobs; (3) each prepared item contains non-empty candidate_profile and job_key; (4) the loop processes each selected job serially; (5) the parser rejects malformed or incomplete model output; (6) Master Job upsert uses the stable job_key; (7) the original Job Intake row is updated in place with status and timestamp; and (8) the next batch fetches remaining eligible rows and stops cleanly when none remain.

37. Closing Assessment: What This Project Demonstrates

MatchIQ is best presented as a digital-transformation project with an AI component, not as an AI demo with a workflow around it. The central work was translating a human decision process into explicit stages, deciding which stages could be deterministic, deciding where AI added genuine value, controlling the cost and rate of inference, creating reusable evidence, preserving lineage, and making failure visible.

The strongest evidence is the sequence of design decisions. The project did not simply add agents. It discovered that three agents were too many, that a second gate rejected almost everything, that a literal experience rule distorted the market, that repeated requests created 429 pressure, that a static source document weakened maintainability, that repeated CV analysis was wasteful, and that a structured-output failure needed a stronger contract.

Each observation changed the system. The architecture therefore tells a coherent project-management story: define the problem, measure the process, identify failure modes, redesign the process, introduce controls, validate the revised behaviour, and preserve the change history.

That is the principal value of MatchIQ as a portfolio project for AI Governance and Digital Transformation roles. It demonstrates the ability to govern AI as a component of a wider business process rather than treating AI as the process itself.

End of data-heavy working draft — MatchIQ Project Case Study v3.0

Current-State Update — 17 September 2026

This update incorporates the latest project-evolution account. It extends the existing case study without reclassifying partially validated integration work as complete. The current dashboard baseline remains approved.

Python ingestion and persistent queue

The Python entry path is main.py → job.py → Google Sheets / Job Intake. The ingestion design discovers and scores opportunities, creates stable job_key values, compares them with existing keys, removes within-run duplicates, and appends up to INGESTION_BATCH_SIZE = 20 unseen jobs. A recorded run found 797 existing keys, selected 15 unseen eligible jobs, and appended them, bringing the data-row count to 812. This demonstrates that the capped selection can return fewer than 20 when fewer unseen eligible jobs are available; it does not guarantee a full batch on every run.

Job Intake is the persistent ingestion queue in the MatchIQ Jobs Input File spreadsheet. Its 55-column schema covers identity/ingestion, discovery/ranking, experience and signals, job detail, and processing state. The intended status fields

include ingestion_status, agent3_status, and processed_at. The queue is distinct from MatchIQ Job Intelligence / Sheet1 and from MatchIQ_Master_Job assessment records.

Environment and integration troubleshooting

The local environment was stabilised on Python 3.13.15 after a Python 3.14 SciPy DLL import failure associated with an Application Control restriction. A project virtual environment was created and SciPy, scikit-learn, and gspread imports were confirmed. Google Sheets export initially failed because pathlib.Path was not imported; adding `from pathlib import Path` resolved that specific error. Service-account authentication, tab/header setup, a controlled append, and a real batch append were validated.

Dashboard implementation and approved baseline

The Next.js dashboard data layer in lib/data.ts fetches /api/jobs with cache: no-store, accepts an array or an object containing a value array, parses Agent 1/Agent 3 assessment fields, and uses the Agent 3 score as the primary fit score with scraper score as fallback. The API route proxies the configured n8n webhook. A curl.exe request to localhost:3000/api/jobs returned JSON, and the build succeeded. The individual Job Analysis page (app/jobs/[id]/page.tsx) was approved after a hydration issue was resolved by restarting the development server in the correct directory. Preserve this approved layout unless a new redesign is requested.

Continuous queue processing — current integration focus

The current goal is one n8n execution that processes successive groups of up to 20 eligible jobs. This requires two distinct loops: (1) item-level serialisation through Loop Over Items (Batch Size 1) and an approximately 40-second Wait to control Groq request pacing; and (2) batch-level continuation that re-reads the persistent Job Intake queue, selects the next eligible jobs, and stops when none remain. These are separate controls.

The latest reported second-pass failure was `No candidate profile found in Merge output.` The first pass and a one-job Agent 3 assessment were successful, but candidate evidence was absent from the Merge input on the subsequent pass. A fallback to the already-executed candidate lookup was proposed; it is not confirmed as a permanent fix.

A queue-reader configuration was observed pointing to MatchIQ Job Intelligence / Sheet1 rather than the intended MatchIQ Jobs Input File / Job Intake. Confirm the live binding. In addition, the original Job Intake row must be updated in place after assessment (agent3_status = SUCCESS and processed_at). Writing a result to MatchIQ_Master_Job does not update the source queue row and therefore cannot alone prevent re-selection.

The remaining acceptance sequence is: complete a batch; persist each outcome to its original queue row; re-read Job Intake; select the next eligible batch; preserve candidate evidence across the continuation; repeat; and terminate cleanly when no eligible rows remain. Failure/resume behaviour also requires validation so completed jobs remain complete while unfinished jobs can be retried safely.

Updated validation status

Area	Status	Evidence / boundary
Python discovery, scoring, deduplication and capped ingestion	Validated in recorded runs	A 15-job unseen batch was appended; 20 is a cap, not a guarantee.
Google Sheets authentication and append path	Validated	Service account, headers, controlled append and batch append tested.
CV retrieval, Agent 2 and candidate cache	Validated in tested path	Full CV retrieved; profile generated, upserted and read back.
Agent 3 structured assessment and Master Job upsert	Validated for tested items	Successful structured output and persistence observed; do not infer full-queue completion.
Dashboard API/data layer and build	Validated	API returned JSON; build succeeded.
Job Analysis page	Approved baseline	User approved the view after hydration issue was resolved.
Continuous multi-batch Agent 3 processing	Integration pending	Second pass lost candidate profile; queue source, source-row status update, continuation and stop/resume

		require end-to-end validation.
--	--	--------------------------------

Hiring-manager narrative — refreshed

MatchIQ has progressed from job-search automation towards an integrated job-intelligence and candidate-role assessment platform. Python now feeds a persistent, deduplicated Google Sheets intake queue; n8n coordinates candidate evidence reuse and controlled role assessments; structured outputs are persisted separately from incoming jobs; and a Next.js dashboard presents the results in an approved recruiter-oriented Job Analysis view. The work demonstrates process redesign, data-layer separation, incremental validation, AI output structuring, rate-control decisions, and translation of technical results into a hiring-manager interface. The principal unfinished integration is reliable continuous queue draining: preserving candidate context across passes, updating original queue rows, and proving repeat/stop/resume behaviour end to end.

EU AI Act — Regulatory Context, Control Mapping and Governance Boundaries

MatchIQ is a functional recruitment and candidate-role assessment prototype. Its intended-use context matters: Annex III point 4(a) of Regulation (EU) 2024/1689 identifies AI systems intended for recruitment or selection, including application analysis/filtering and candidate evaluation, as high-risk use cases, subject to the Act's scope and applicable conditions. This section maps relevant themes to MatchIQ while separating implemented design features from obligations and controls not yet demonstrated.

This is a governance-oriented design analysis, not a legal conformity assessment. The prototype must not be described as EU AI Act compliant, audit-ready, or as having implemented every high-risk obligation.

1. Regulatory classification and project relevance

The supplied material points to the correct regulatory concern, but the relevant Annex III citation is point 4(a), not paragraph 1(b). The precise classification and obligations depend on the system's intended purpose, deployment context, and applicable provisions.

2. Design-to-governance mapping

Regulatory theme	MatchIQ design evidence	Governance relevance	Boundary / remaining work
Article 12 — Record-keeping / logging	Stable job_key values, persistent Job Intake records, processing statuses, separate Master Job assessment records, and structured outputs.	Supports identification of records and separation of discovery data from candidate-role assessments.	These features do not by themselves demonstrate automatic event logging, completeness, retention, integrity, or a full audit trail. Specify and validate event coverage, timestamps, system/model/configuration versions, errors, retention and access controls.
Article 11 and Annex IV — Technical documentation	Documented Python → Google Sheets → n8n → Groq/Agent 2 and Agent 3 → Master Job → Next.js flow; structured JSON Schema outputs; component and validation records.	Provides a foundation for documenting intended purpose, boundaries, data flows, model use and output contracts.	Structured JSON is not a substitute for Annex IV documentation. Complete and maintain the applicable technical file, including system description, development/validation methods, risk management, data governance, monitoring and conformity materials.
Article 14 — Human oversight	The Job Analysis dashboard surfaces fit evidence, transferable experience, gaps, risks and hiring-manager validation points.	Designed to support human review rather than show only an opaque score.	A dashboard alone does not prove effective oversight or prevent automation bias. Define reviewer competence, authority, intervention/override and escalation, meaningful review, instructions and safeguards against solely automated consequential decisions.

3. What MatchIQ can credibly demonstrate

- A recruitment-related AI prototype designed with awareness of its high-risk regulatory context.
- Separation of deterministic job discovery/prioritisation from candidate-specific AI assessment.

- Stable identifiers and persistent records that support data continuity and potential traceability.
- Structured outputs that make assessment fields easier to validate, store, inspect and present consistently.
- A recruiter-facing interface designed to expose evidence and validation points for human review.
- Explicit recognition that prototype features do not, by themselves, establish legal compliance.

4. Controls not yet established for a compliant deployment

- Formal applicability/conformity assessment, including provider/deployer roles and actual intended purpose.
- Demonstrated Article 12 logging and record-keeping, including event coverage, retention, integrity and review procedures.
- Complete Article 11 / Annex IV technical documentation and controlled versioning.
- An Article 14 oversight operating model with trained reviewers, authority to intervene, override/escalation procedures and automation-bias safeguards.
- Risk management, data governance, accuracy, robustness, cybersecurity, quality management, monitoring and other obligations applicable to the system and deployment.
- End-to-end validation of continuous queue processing, original-row status updates, safe retries/resume and clean termination.

5. Interview framing

“I designed MatchIQ as a recruitment-related AI automation prototype with the EU AI Act’s high-risk context in view. I separated deterministic job prioritisation from candidate-role assessment, used persistent identifiers and structured outputs to support traceability and validation, and designed the recruiter dashboard to expose evidence, gaps, risks and human-review points. I also documented the boundary between prototype design controls and the additional logging, technical documentation, oversight and operational controls needed before any compliance claim.”

Avoid claiming that MatchIQ is “EU AI Act compliant,” “audit-ready,” or that it “eliminates automation bias.” Present these as design intentions or governance themes unless the corresponding controls have been implemented and evidenced.

Reference

Regulation (EU) 2024/1689 (EU AI Act): Annex III point 4(a); Article 11 and Annex IV (technical documentation); Article 12 (record-keeping); Article 14 (human oversight). This project-document mapping is not legal advice; check the consolidated text and deployment-specific requirements before making external compliance claims.

Change Management Addition — High-Match Keyword Signatures

Change rationale: the discovery stage was spending enrichment effort on a broad set of search-card results, even though the product objective is to surface roles likely to be strong matches. The change uses recurring keyword and phrase patterns observed in high-match job descriptions to estimate which newly discovered roles are worth enriching with the full job description. This is a resource-allocation step before full deterministic MatchIQ scoring—not a replacement for that scoring or the candidate-specific Agent 3 assessment.

What changed

- Introduced configurable high-match keyword signatures and a prediction/ranking step over discovered search cards.
- Added per-country and per-family enrichment caps, plus a minimum predicted top-match threshold, to control detail-endpoint usage.
- Enriches the shortlisted candidates with full job descriptions, then applies full MatchIQ scoring and final eligibility/tiering.
- Keeps the Opportunity Gate diagnostic-only so it does not become a second rejection filter.

Why this design

High-match jobs provide a practical source of observable language patterns—such as recurring capability, functional, domain, governance, transformation, seniority and responsibility signals. Using those patterns at the search-card stage helps prioritise likely-relevant roles before making more expensive or rate-limited detail requests. Country and job-family caps reduce the risk that one geography or role family consumes the entire enrichment budget.

The predictor is intentionally a shortlist heuristic. Search-card text can be incomplete, keywords can be ambiguous, and relevant hybrid roles may use unexpected titles or wording. Therefore, predicted high-match status is not a final match verdict: full-description scoring and the separate candidate-role assessment remain downstream checks. Thresholds and keyword signatures should be versioned and evaluated against observed outcomes to monitor missed strong matches and unnecessary enrichment.

Change record

Change	Problem / trigger	Implementation	Validation / control
High-match keyword-based pre-enrichment shortlist	Broad discovery results made detail enrichment costly and vulnerable to endpoint/rate limits; objective is to spend enrichment capacity on roles with stronger predicted relevance.	Inspect high-match job language; encode keyword/phrase signatures; predict and rank search-card candidates; apply configurable minimum prediction and per-country/per-family caps; enrich shortlisted jobs; then run full MatchIQ scoring.	Treat prediction as prioritisation only. Validate precision and missed-match rate against full-JD outcomes; version signatures and thresholds; retain final scoring and explicit experience/quality checks. Opportunity Gate remains diagnostic.

Current implementation boundary

The supplied current-state code describes `HIGH_MATCH_KEYWORD_SIGNATURES`, `predict_top_match()`, and `rank_discovery_for_enrichment()`, with configurable `MIN_TOP_MATCH_PREDICTION`, `MAX_ENRICHMENT_PER_COUNTRY`, and `MAX_ENRICHMENT_PER_FAMILY`. It ranks discovered jobs, enriches the resulting shortlist, and only then applies full scoring and filtering. The predictor's accuracy has not been established by a documented benchmark in the supplied evidence; describe it as a heuristic designed to improve enrichment allocation, not as proven to return only high matches.

EU AI Act Governance Framing: Control Evidence and Deployment Boundaries

MatchIQ is documented as a recruitment-adjacent decision-support prototype whose intended use may engage the high-risk context described in Regulation (EU) 2024/1689, Annex III, point 4(a). This is a design and governance mapping, not a determination of legal classification, conformity, or compliance. Applicability depends on the system's intended purpose, deployment context, and the Act's applicable provisions.

Control principle: evidence, not assertion

The governance claim is deliberately bounded: documented technical mechanisms can support traceability, structured review, and separation of responsibilities, but they do not by themselves prove that MatchIQ satisfies the EU AI Act. Each control is therefore described in three parts: what is evidenced in the prototype, what that evidence does and does not establish, and what remains an operational deployment target.

Control-to-evidence ledger

Regulatory theme	Prototype evidence described	What this supports (bounded)	Operational target / evidence still required
Article 12 — record-keeping and logging	Stable job_key generation; persistent job records; processing statuses; source and result data separated across intake and master records.	Supports record identity, deduplication, and partial data lineage across the documented workflow.	Do not call the key immutable or the logs complete unless technically demonstrated. Add and validate event-level logs for execution IDs, timestamps, input/output references, prompt/model/config versions, errors, retries, and relevant system changes; define retention, access, integrity, and export controls.
Article 14 — human oversight	Next.js Job Analysis view exposes structured fit information, evidence, gaps/risks, transferable experience, and human validation checkpoints.	Makes model output more inspectable than a single opaque score and provides material for human review.	A UI alone does not prove effective oversight or prevent automation bias. Define reviewer authority, training, escalation, override and contestation routes, monitoring for over-reliance, and evidence that reviewers can understand and challenge outputs.
Article 11 and Annex IV — technical documentation	The project document records architecture, component roles, model configuration, schemas, change history, and known limitations where evidenced.	Provides a useful starting index for a technical documentation package.	This case study is not a complete Annex IV technical file. Maintain version-controlled system description, intended purpose, risk management, data governance, validation, performance, limitations, monitoring, and other required technical evidence.
Data governance / minimisation foundations	Agent 2 is designed to produce a reusable CV evidence profile without vacancy context; Agent 3	Supports separation of candidate evidence extraction from vacancy-specific assessment and	This does not prove absence of bias, hallucination, or data leakage. Validate extraction accuracy and

	performs job-specific comparison. Candidate profile caching avoids repeated extraction when the source version is unchanged.	may reduce unnecessary repeated processing.	cache invalidation; document lawful basis, purpose limitation, data quality/representativeness, retention, access, data-subject processes, and bias testing.
External inference and data boundary	The MVP calls an external Groq endpoint using the documented model route.	Makes the inference dependency visible as an architectural boundary.	Assess data categories sent, provider terms, location, retention, subprocessors, security, transfer mechanisms, and contractual controls. EEA-local or otherwise approved inference is a target-state option, not an implemented MVP capability.

Article 12: traceability claim — precise wording

The prototype uses stable job identifiers and persistent records to support linkage between discovered vacancies, processing state, and stored assessments. These mechanisms improve record-level traceability. They should not be described as an immutable, end-to-end audit log: the documented implementation does not establish complete system-event telemetry, tamper resistance, or capture of every prompt, environment, reviewer, and configuration change.

Article 14: oversight claim — precise wording

The dashboard is designed to support human review by presenting structured evidence, gaps, risks, and validation points rather than relying solely on a score. Human review remains the intended decision authority in the prototype. Effective oversight is an operational property that must be demonstrated through reviewer procedures, training, challenge/override capability, and monitoring; the interface alone cannot establish that automation bias has been eliminated or that a recruiter must review every result.

Vacancy-independent candidate evidence: control and limitation

Agent 2's vacancy-independent extraction is a separation-of-concerns design: the candidate evidence profile is generated from the CV without job-description parameters, then reused by the job-specific Agent 3 assessment. This reduces one route for vacancy-specific contamination of the extracted profile. It does not guarantee that the extraction is error-free, unbiased, immutable, or incapable of omission. Source fidelity, versioning, correction, and cache invalidation remain validation requirements.

Current-state versus target-state register

Area	Current prototype boundary	Target-state acceptance evidence
Logging	Stable IDs, records, statuses and timestamps where captured; completeness and tamper resistance not established.	End-to-end event schema, correlated execution IDs, integrity controls, retention/access policy, tested log export and reconciliation.
Human oversight	Dashboard exposes structured assessments and validation points; operational reviewer controls not proven.	Approved SOP, named decision authority, reviewer training, override/escalation workflow, sampling and automation-bias monitoring.
Queue reliability	Queue binding, in-place status update, multi-batch continuity and safe resumption require validation as identified in the project record.	Acceptance tests demonstrating correct source sheet, idempotent updates, no duplicate reprocessing, and recovery after mid-run failure.
Model and prompt governance	Configuration and schema are	Versioned prompt/model/config

	documented at the level available in the project record; full change telemetry is not established.	registry, approval workflow, regression suite, release records, rollback and post-release monitoring.
Data protection and deployment	External Groq inference is part of the MVP; EEA/local inference and enterprise controls are not implemented claims.	Documented data-flow assessment, provider/transfer review, retention and access controls, security testing, and approved deployment architecture.

Interview positioning: defensible version

“I used the EU AI Act’s high-risk framework as a design and governance reference, rather than treating compliance as a marketing claim. In MatchIQ, stable job identifiers, persistent records, and processing statuses support record-level traceability, while the dashboard presents structured fit evidence, gaps, and validation points to support human review. I also separated vacancy-independent CV evidence extraction from job-specific assessment. I documented the limits of those controls: the prototype does not yet demonstrate complete tamper-resistant Article 12 logging, effective Article 14 oversight in operation, or full conformity. The case study distinguishes implemented mechanisms from the logging, reviewer, data-governance, validation, and deployment controls that would need to be completed and evidenced before an enterprise deployment.”

Terminology guardrails for the case study

- Prefer “supports traceability” over “proves Article 12 adherence” unless complete, validated logging evidence exists.
- Prefer “designed to support human oversight and mitigate automation-bias risk” over “eliminates automation bias” or “forces human review.”
- Describe job_key as “stable” or “deterministic” only to the extent established by the implementation; do not call it “immutable” without integrity controls.
- Describe the system as a “prototype designed with awareness of the high-risk context,” not “EU AI Act compliant,” “certified,” or “audit-ready.”
- Keep the external Groq inference boundary and EEA/local inference as a target-state consideration, not a current capability.
- Treat Annex III, point 4(a) as a potentially relevant use-case context; confirm formal classification against intended purpose and deployment facts.

Current-State Update — Lean Adjacent-Role Calibration v4

This update records the supplied scraper source dated 20 September 2026. It extends the previous case-study snapshot; it does not imply that the new configuration has been run end-to-end or that its retrieval precision, recall, or hiring outcomes have been validated.

1. Expanded European search coverage

The current script expands its configured search markets from the previously documented six-country set to 15 countries: Luxembourg, Germany, Italy, Spain, Switzerland, Poland, Malta, the Netherlands, Ireland, France, Russia, Ukraine, Romania, the Czech Republic and the United Kingdom. Search anchors remain city-based within each market. This broadens geographic discovery, but configured coverage is not evidence of equal vacancy availability, language accessibility, legal eligibility, or successful retrieval in every country.

2. Adjacent-role taxonomy and multidimensional signals

The script is designed to discover roles beyond exact “AI Governance” titles. Its taxonomy and signal sets cover governance, transformation, financial crime, product/programme, regulatory and stakeholder fit, while also surfacing language requirements, international-hiring markers, scarce-specialist signals, engineering-heavy roles and legal-counsel signals. English and selected local-language variants are included in query generation. These are discovery and deterministic-scoring signals—not proof of role suitability or candidate employability.

3. Pre-enrichment shortlist and request-budget controls

The updated pipeline applies a lean adjacent-role calibration before expensive full-description enrichment. It uses `HIGH_MATCH_KEYWORD_SIGNATURES` with `predict_top_match()` and `rank_discovery_for_enrichment()` to rank search-card results, then enriches only the selected shortlist before full scoring and filtering. The supplied configuration sets `MIN_TOP_MATCH_PREDICTION=14`, `MAX_ENRICHMENT_PER_COUNTRY=50` and `MAX_ENRICHMENT_PER_FAMILY=10`. The ingestion batch size is 20. These are configured limits, not a guarantee that 20 jobs—or any fixed number—will be appended in a run.

The code also sets `MAX_JOBS_PER_QUERY=10`, `PAGES_PER_QUERY=1`, `REQUEST_DELAY=1.5` seconds, `REQUEST_TIMEOUT=20` seconds, `MAX_RETRIES=4` and a 30-day lookback parameter. A seven-day detail-cache TTL is configured. These controls aim to constrain request volume and reuse recent detail data; they do not guarantee avoidance of HTTP 429 responses, cache correctness, or compliance with LinkedIn terms. The source itself flags that guest endpoints can change or rate-limit requests and instructs operators to respect applicable terms and laws.

4. Experience and job-quality rules

The source configures a target experience range of 3–10 years, requires a full job description, and retains separate experience extraction/classification logic. It also defines hard-excluded title terms and an opportunity-gate threshold of 18, with a final-match minimum of 0. These values describe current code configuration; they should be checked against the intended candidate profile and monitored for exclusion bias, especially across seniority and adjacent-role families.

5. Persistent intake and deduplication

The scraper is configured to write to the Google Sheets “Job Intake” tab, with a stable job identity key, existing-key loading and append-only handling of newly unseen jobs. The supplied code contains a defined Job Intake header schema and Google Sheets client/append functions. This supports incremental intake and reduces repeat rows, but it is not evidence of a successful run against the live sheet in this update, nor does it establish immutable audit logging.

6. Change impact and validation status

Change record: CHG-018 (proposed documentation ID) — broaden geographic coverage and adjacent-role discovery; introduce calibrated pre-enrichment ranking and explicit country/family request caps; tune experience and request controls; retain persistent Google Sheets intake. Change class: B (logic) with A (configuration) and E (data/terms considerations) implications. The identifier is assigned here for traceability and should be reconciled with the project's authoritative change register.

Validation status: source inspected; runtime execution, live-sheet write verification, regression tests, rate-limit behaviour, cache hit/miss behaviour, and measured shortlist quality were not demonstrated by the source file alone. In particular, the keyword predictor is a heuristic over incomplete search-card text. Its threshold and signature set require evaluation against full-JD outcomes, including false inclusions and missed strong matches. A lexical score must not be described as a probability of being hired.

7. Recommended acceptance checks for this change

- Run a controlled sample across several countries and role families; record discovered, shortlisted, enriched, eligible and appended counts.
- Measure predictor precision and recall against full-description scoring and human-reviewed labels; inspect false positives and false negatives.
- Confirm country and family caps behave as intended when the same vacancy appears across cities or queries.
- Verify stable job-key deduplication and append-only behaviour against the live Job Intake sheet, including rerunning the same sample.
- Record HTTP status distribution, retries, 429 frequency, cache hits, runtime and request count.
- Review local-language and work-authorisation requirements separately; do not infer legal eligibility from country targeting.
- Preserve a versioned configuration snapshot and rollback path before production use.

Interview-ready explanation

“I broadened MatchIQ from exact-title discovery to a calibrated adjacent-role search across 15 European markets. The scraper now ranks search-card evidence before requesting full job descriptions, with country- and role-family caps to constrain enrichment volume. It then applies full-description scoring and persists unseen opportunities to a Google Sheets intake queue using stable identity keys. I treat the pre-enrichment score as a prioritisation heuristic—not a hiring probability—and the next validation step is to benchmark its precision, recall, rate-limit behaviour and deduplication against controlled runs.”